

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Cache and Memory Hierarchy (Part 3)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Mikko Lipasti

Last Lecture

- Cache architecture
- Cache misses
- False sharing

This Lecture

- Parallelism and cache locality
- Loop cost model
- Approximate caching

Loop Dependences

Are these loops parallelizable?

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        a[i][j] = a[i][j-1];  
        ... = b[i][j];  
    }  
}
```

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        c[i][j] = c[i][j-1];  
        b[(n-1)-i][j] = c[i][j];  
    }  
}
```

Loop Dependences

Are these loops parallelizable?

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {

    for(int j = 0; j < n; j++) {
        a[i][j] = a[i][j-1];
        ... = b[i][j];
    }
}
```

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {

    for(int j = 0; j < n; j++) {
        c[i][j] = c[i][j-1];
        b[(n-1)-i][j] = c[i][j];
    }
}
```

Loop Dependences

Are these loops parallelizable?

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {

    for(int j = 0; j < n; j++) {
        a[i][j] = a[i][j-1];
        ... = b[i][j];
    }
}

#pragma omp parallel for
for(int i = 0; i < n; i++) {

    for(int j = 0; j < n; j++) {
        c[i][j] = c[i][j-1];
        b[(n-1)-i][j] = c[i][j];
    }
}
```

*Are there any concerns
in performance?*

Parallelism and Cache Locality

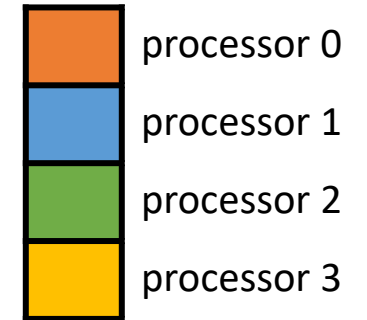
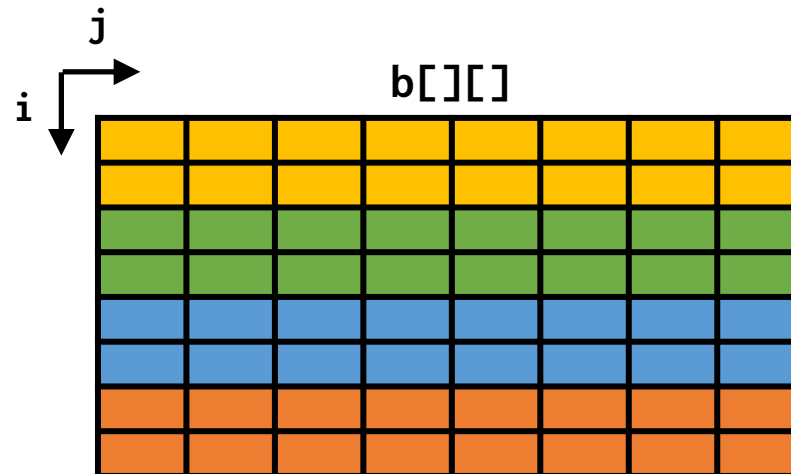
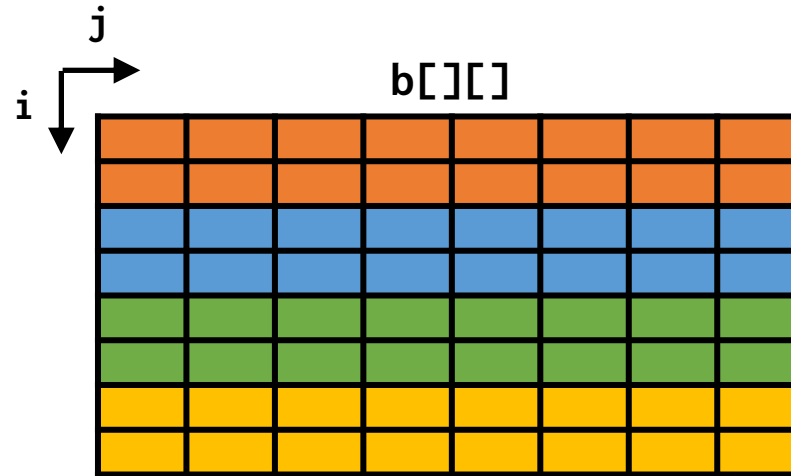
Are these loops parallelizable?

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {

    for(int j = 0; j < n; j++) {
        a[i][j] = a[i][j-1];
        ... = b[i][j];
    }
}

#pragma omp parallel for
for(int i = 0; i < n; i++) {

    for(int j = 0; j < n; j++) {
        c[i][j] = c[i][j-1];
        b[(n-1)-i][j] = c[i][j];
    }
}
```



Parallelism and Cache Locality

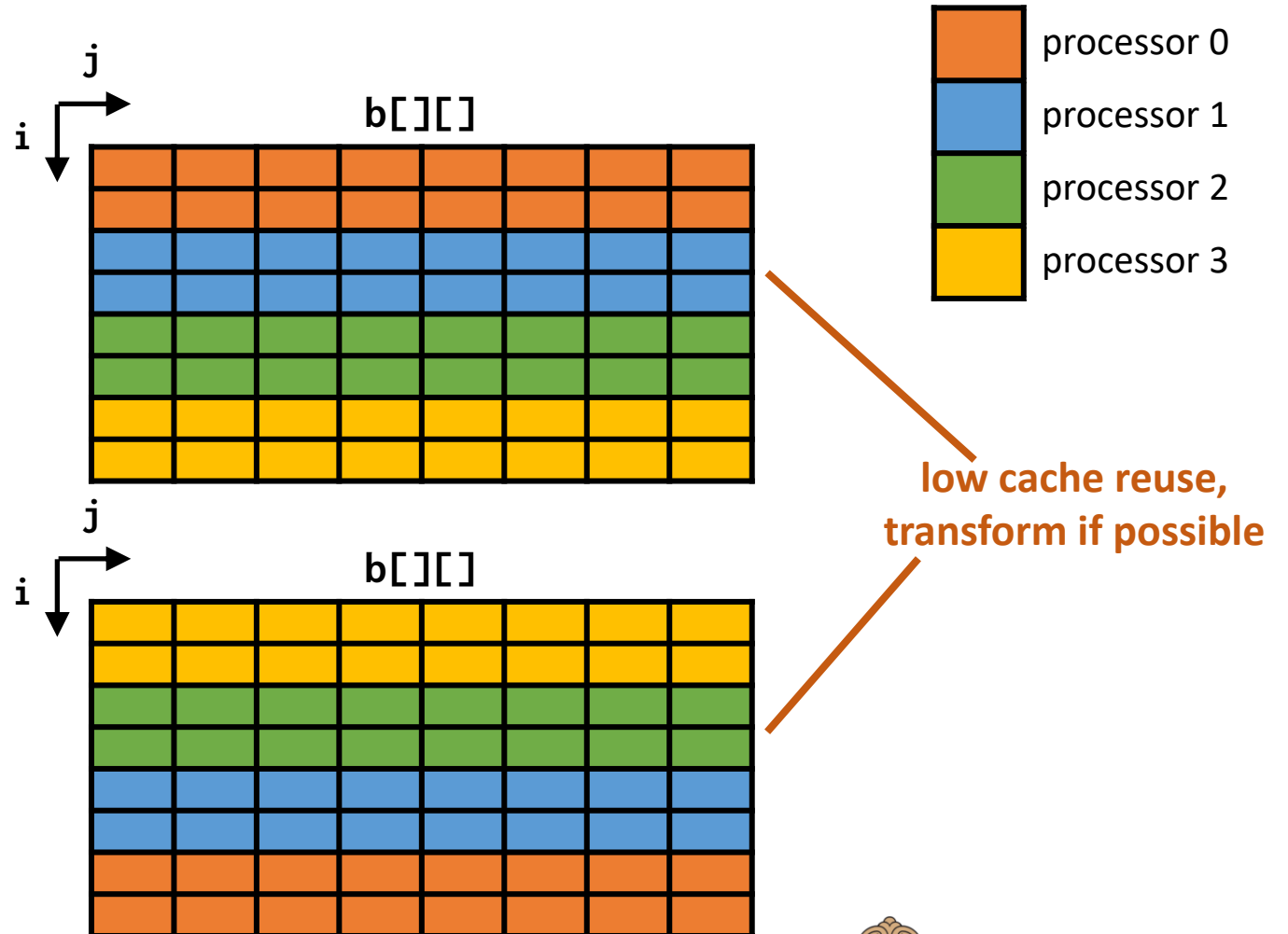
Are these loops parallelizable?

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {

    for(int j = 0; j < n; j++) {
        a[i][j] = a[i][j-1];
        ... = b[i][j];
    }
}

#pragma omp parallel for
for(int i = 0; i < n; i++) {

    for(int j = 0; j < n; j++) {
        c[i][j] = c[i][j-1];
        b[(n-1)-i][j] = c[i][j];
    }
}
```



Parallelism and Cache Locality

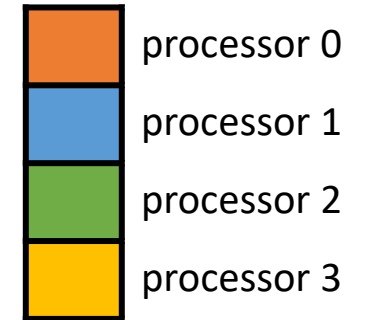
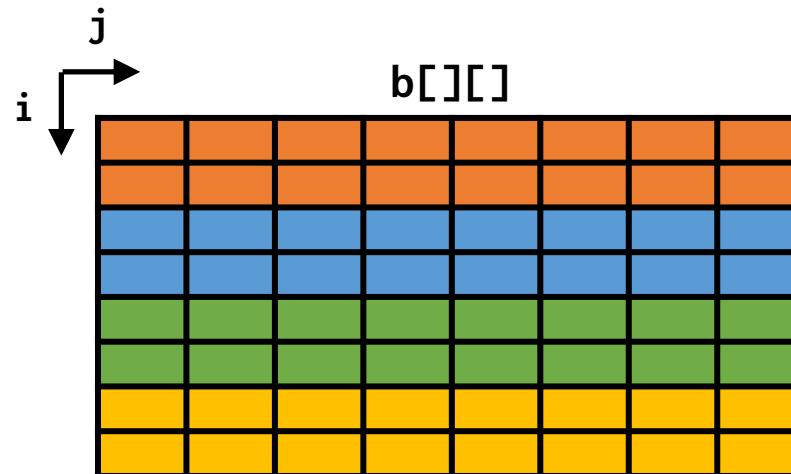
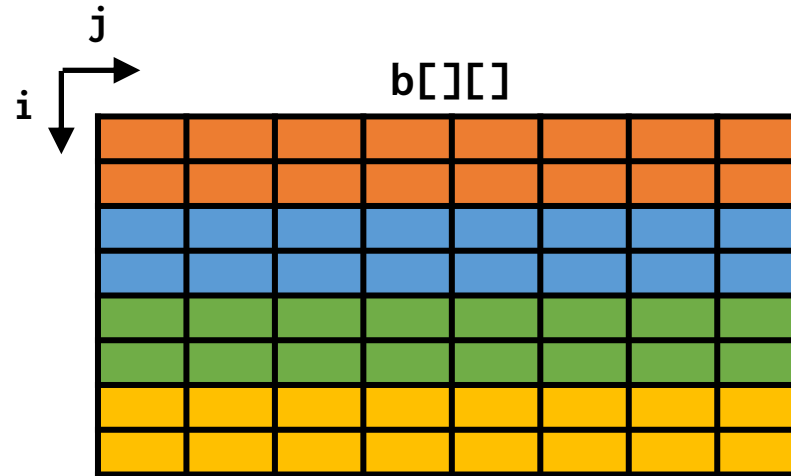
Are these loops parallelizable?

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {

    for(int j = 0; j < n; j++) {
        a[i][j] = a[i][j-1];
        ... = b[i][j];
    }
}

#pragma omp parallel for
for(int i = n-1; i >= 0; i--) {

    for(int j = 0; j < n; j++) {
        c[i][j] = c[i][j-1];
        b[(n-1)-i][j] = c[i][j];
    }
}
```



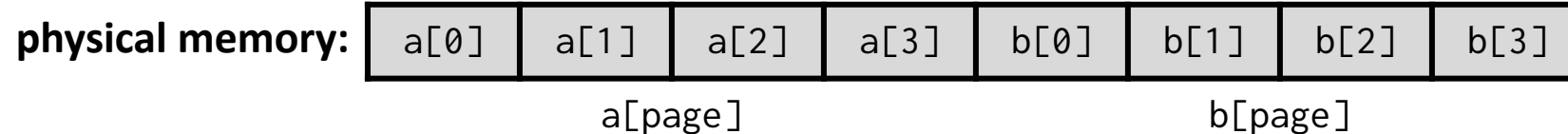
Parallelism and Cache Locality

Assume 2 processors and 4KB pages:

```
char a[16384]; // 4 pages
```

```
char b[16384]; // 4 pages
```

```
#pragma omp parallel for  
for(i = 0; i < 16384; i++)  
    a[i] = ... b[i];
```



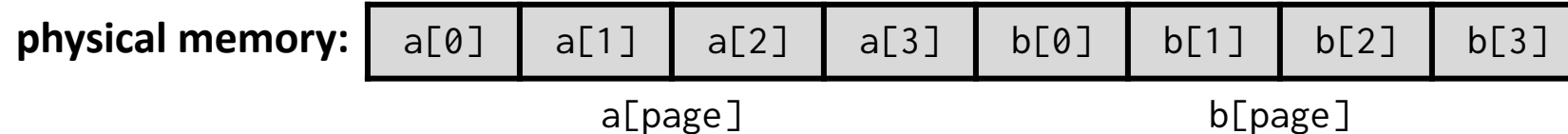
Parallelism and Cache Locality

Assume 2 processors and 4KB pages:

```
char a[16384]; // 4 pages  
char b[16384]; // 4 pages
```

```
#pragma omp parallel for  
for(i = 0; i < 16384; i++)  
    a[i] = ... b[i];
```

What if 16KB private L1 data caches?



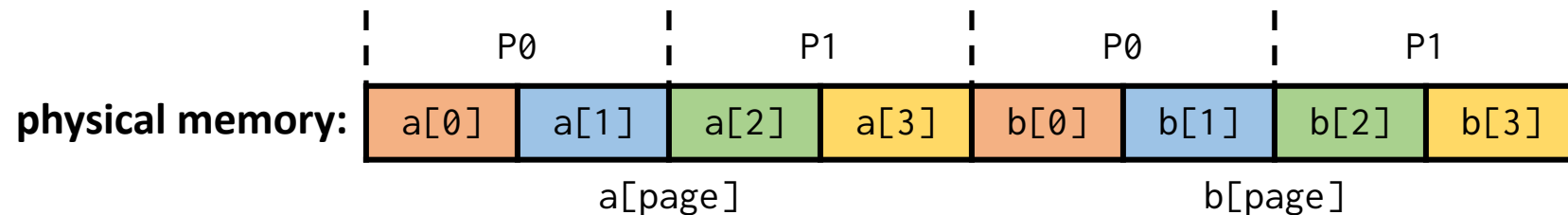
Parallelism and Cache Locality

Assume 2 processors and 4KB pages:

```
char a[16384]; // 4 pages  
char b[16384]; // 4 pages
```

```
#pragma omp parallel for  
for(i = 0; i < 16384; i++)  
    a[i] = ... b[i];
```

What if 16KB private L1 data caches?



L1 cache sets



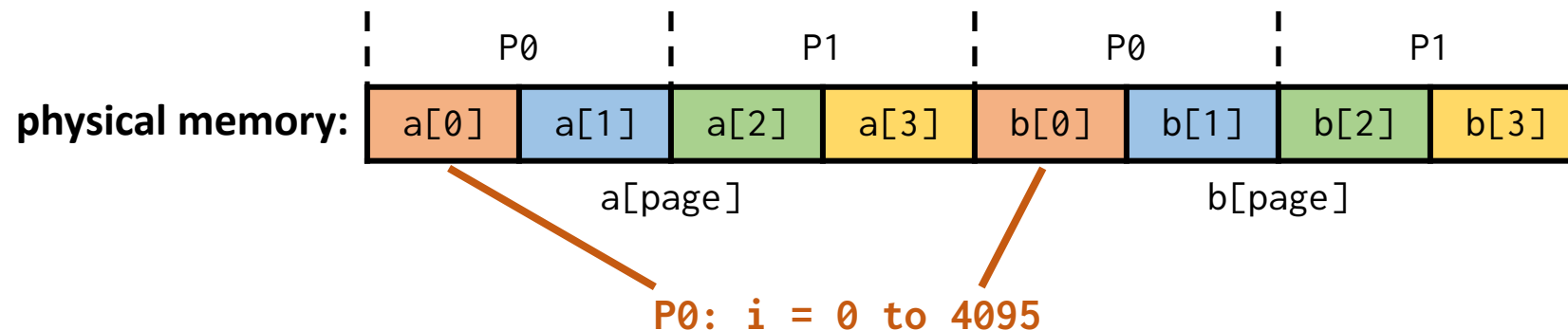
Parallelism and Cache Locality

Assume 2 processors and 4KB pages:

```
char a[16384]; // 4 pages  
char b[16384]; // 4 pages
```

```
#pragma omp parallel for  
for(i = 0; i < 16384; i++)  
    a[i] = ... b[i];
```

What if 16KB private L1 data caches?



L1 cache sets



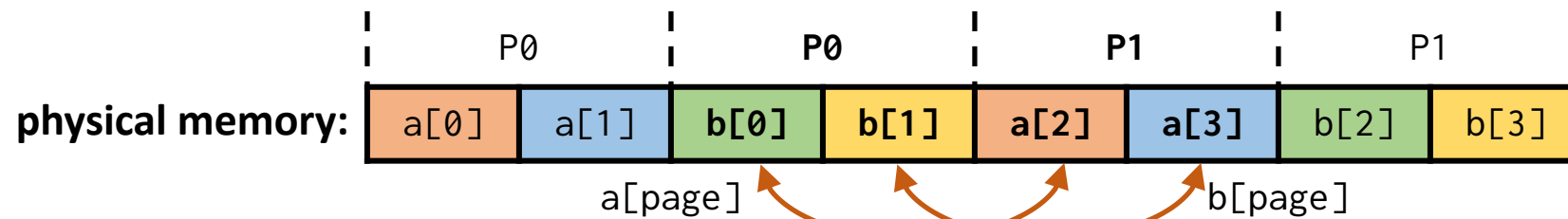
Parallelism and Cache Locality

Assume 2 processors and 4KB pages:

```
char a[16384]; // 4 pages  
char b[16384]; // 4 pages
```

```
#pragma omp parallel for  
for(i = 0; i < 16384; i++)  
    a[i] = ... b[i];
```

What if 16KB private L1 data caches?



swap in physical memory via page coloring
[E. Bugnion et al., ASPLOS 1996]

L1 cache sets



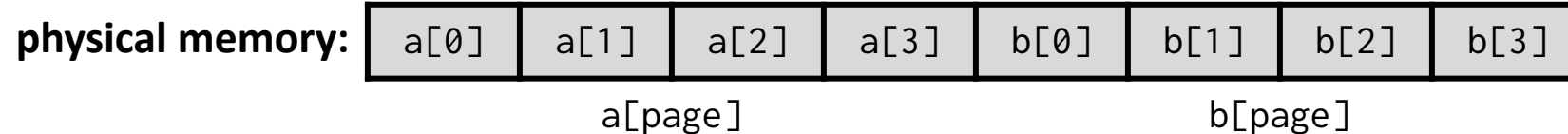
Parallelism and Cache Locality

Assume 2 processors and 4KB pages:

```
char a[16384]; // 4 pages  
char b[16384]; // 4 pages
```

```
#pragma omp parallel for  
for(i = 0; i < 16384; i++)  
    a[i] = ... b[i];
```

What if 8KB private L1 data caches?



Parallelism and Cache Locality

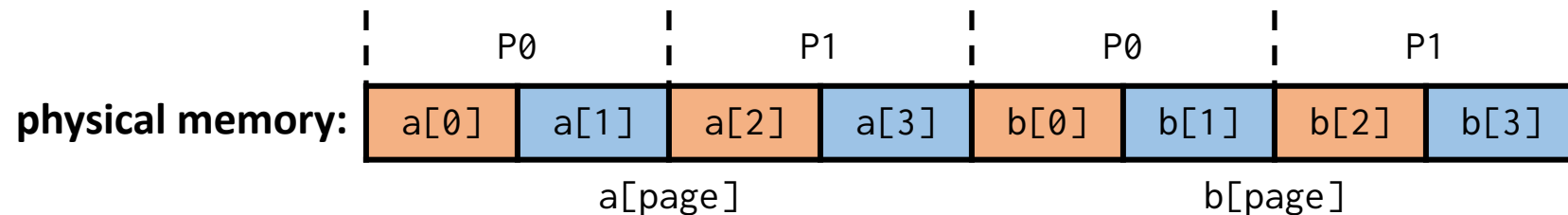
Assume 2 processors and 4KB pages:

```
char a[16384]; // 4 pages  
char b[16384]; // 4 pages
```

```
#pragma omp parallel for  
for(i = 0; i < 16384; i++)  
    a[i] = ... b[i];
```

What if 8KB private L1 data caches?

L1 cache sets



Parallelism and Cache Locality

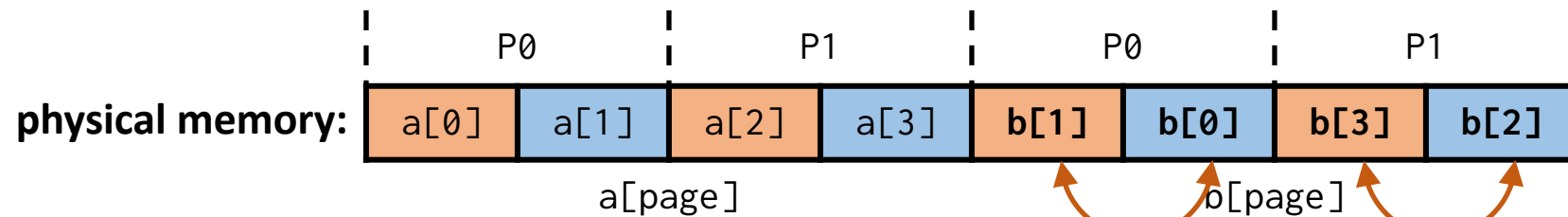
Assume 2 processors and 4KB pages:

```
char a[16384]; // 4 pages  
char b[16384]; // 4 pages
```

```
#pragma omp parallel for  
for(i = 0; i < 16384; i++)  
    a[i] = ... b[i];
```

What if 8KB private L1 data caches?

L1 cache sets



Parallelism and Cache Locality

parallel inner loop

```
for (i = 0; i < N-1; i++) {  
    #pragma omp parallel for  
    for (j = 0; j < N; j++) {  
        ...  
        X[i][j] = f( X[i-1][j] );  
        ...  
    }  
}
```

VS

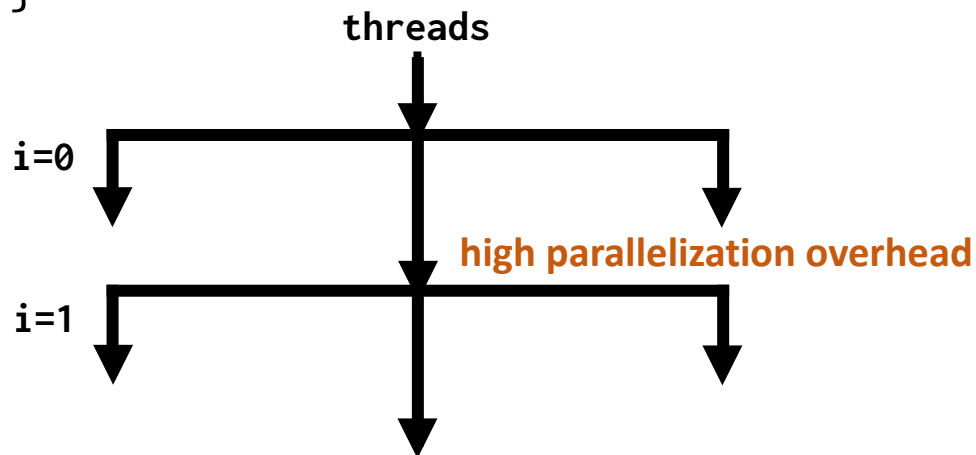
parallel outer loop

```
#pragma omp parallel for  
for (j = 0; j < N; j++) {  
    for (i = 0; i < N-1; i++) {  
        ...  
        X[i][j] = f( X[i-1][j] );  
        ...  
    }  
}
```

Parallelism and Cache Locality

parallel inner loop

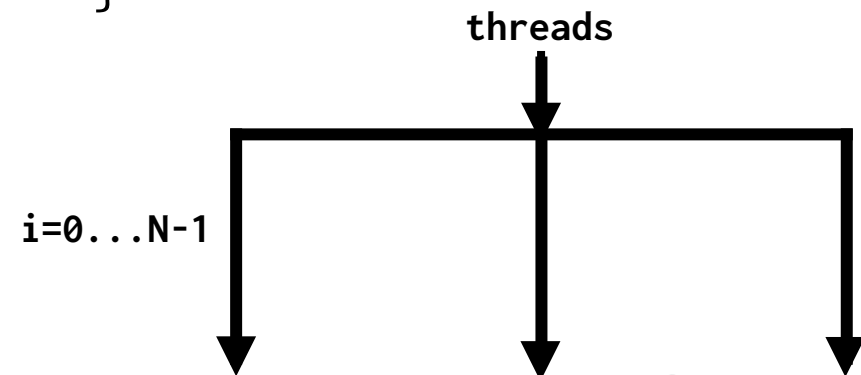
```
for (i = 0; i < N-1; i++) {  
    #pragma omp parallel for  
    for (j = 0; j < N; j++) {  
        ...  
        X[i][j] = f( X[i-1][j] );  
        ...  
    }  
}
```



VS

parallel outer loop

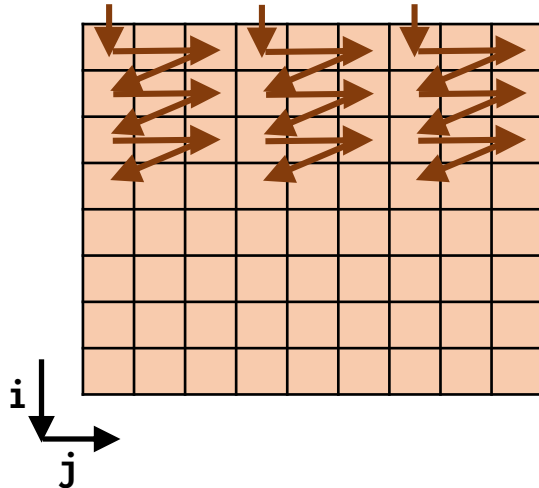
```
#pragma omp parallel for  
for (j = 0; j < N; j++) {  
    for (i = 0; i < N-1; i++) {  
        ...  
        X[i][j] = f( X[i-1][j] );  
        ...  
    }  
}
```



Parallelism and Cache Locality

parallel inner loop

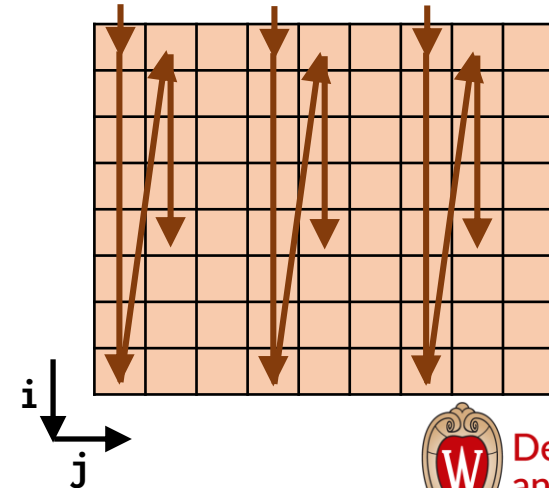
```
for (i = 0; i < N-1; i++) {  
  #pragma omp parallel for  
  for (j = 0; j < N; j++) {  
    ...  
    X[i][j] = f( X[i-1][j] );  
    ...  
  }  
}
```



VS

parallel outer loop

```
#pragma omp parallel for  
for (j = 0; j < N; j++) {  
  for (i = 0; i < N-1; i++) {  
    ...  
    X[i][j] = f( X[i-1][j] );  
    ...  
  }  
}
```



low cache locality



Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- Estimate which loop would have the least number of cache misses if it were placed innermost
- Simplifying assumption: Each array dimension is very large
- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        for (k = 0; k < N; k++)
            c[i][j] += a[i][k] * b[k][j];
```

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```

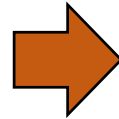
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop					
j loop					
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (i = 0; i < N; i++)
      c[i][j] += a[i][k] * b[k][j];
```

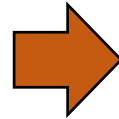
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop					
j loop					
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (i = 0; i < N; i++)
      c[i][j] += a[i][k] * b[k][j];
```

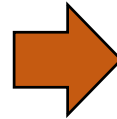
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N				
j loop					
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (i = 0; i < N; i++)
      c[i][j] += a[i][k] * b[k][j];
```

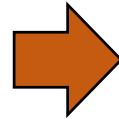
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N			
j loop					
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (i = 0; i < N; i++)
      c[i][j] += a[i][k] * b[k][j];
```

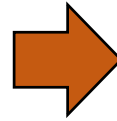
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1		
j loop					
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (i = 0; i < N; i++)
      c[i][j] += a[i][k] * b[k][j];
```

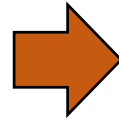
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	2N + 1	
j loop					
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (i = 0; i < N; i++)
      c[i][j] += a[i][k] * b[k][j];
```

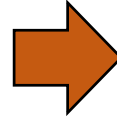
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop					
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (j = 0; j < N; j++)
      c[i][j] += a[i][k] * b[k][j];
```

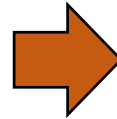
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop					
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (j = 0; j < N; j++)
      c[i][j] += a[i][k] * b[k][j];
```

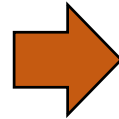
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop	N/L				
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (j = 0; j < N; j++)
      c[i][j] += a[i][k] * b[k][j];
```

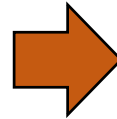
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop	N/L	1			
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (j = 0; j < N; j++)
      c[i][j] += a[i][k] * b[k][j];
```

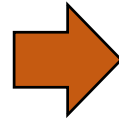
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop	N/L	1	N/L		
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (j = 0; j < N; j++)
      c[i][j] += a[i][k] * b[k][j];
```

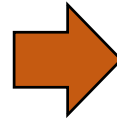
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop	N/L	1	N/L	$2N/L + 1$	$2N^3/L + N^2$
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```

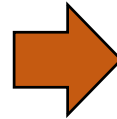
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop	N/L	1	N/L	$2N/L + 1$	$2N^3/L + N^2$
k loop					

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```

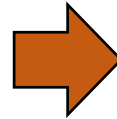
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop	N/L	1	N/L	$2N/L + 1$	$2N^3/L + N^2$
k loop	1				

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```

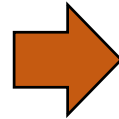
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop	N/L	1	N/L	$2N/L + 1$	$2N^3/L + N^2$
k loop	1	N/L			

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```

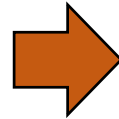
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop	N/L	1	N/L	$2N/L + 1$	$2N^3/L + N^2$
k loop	1	N/L	N		

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k?

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```

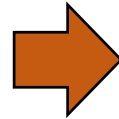
innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop	N/L	1	N/L	$2N/L + 1$	$2N^3/L + N^2$
k loop	1	N/L	N	$(L+1)N/L + 1$	$(L+1)N^3/L + N^2$

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j, k? **j loop**

```
#pragma omp parallel for
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];
```



```
#pragma omp parallel for
for (...)
  for (...)
    for (j = 0; j < N; j++)
      c[i][j] += a[i][k] * b[k][j];
```

innermost	cache misses in inner loop			inner loop cost	total loop cost
	c[i][j]	a[i][k]	b[k][j]		
i loop	N	N	1	$2N + 1$	$2N^3 + N^2$
j loop	N/L	1	N/L	$2N/L + 1$	$2N^3/L + N^2$
k loop	1	N/L	N	$(L+1)N/L + 1$	$(L+1)N^3/L + N^2$

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j?

```
for (i = 0; i < N; i++)  
    for (j = 1; j < N-1; j++)  
        a[j][i] = b[i][j-1] + b[i][j] + b[i][j+1] + c[j][i];
```

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j?

```
for (j = 1; j < N-1; j++)  
  for (i = 0; i < N; i++)  
    a[j][i] = b[i][j-1] + b[i][j] + b[i][j+1] + c[j][i];
```

innermost	inner loop cost	total loop cost
i loop		
j loop		

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j?

```
for (j = 1; j < N-1; j++)  
  for (i = 0; i < N; i++)  
    a[j][i] = b[i][j-1] + b[i][j] + b[i][j+1] + c[j][i];  
      N/L
```

innermost	inner loop cost	total loop cost
i loop		
j loop		

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j?

```
for (j = 1; j < N-1; j++)  
  for (i = 0; i < N; i++)  
    a[j][i] = b[i][j-1] + b[i][j] + b[i][j+1] + c[j][i];  
           N/L           N
```

innermost	inner loop cost	total loop cost
i loop		
j loop		

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j?

```
for (j = 1; j < N-1; j++)  
  for (i = 0; i < N; i++)  
    a[j][i] =  $\underbrace{b[i][j-1] + b[i][j] + b[i][j+1]}_N + c[j][i];$   
               $\underbrace{\hspace{1cm}}_{N/L}$ 
```

innermost	inner loop cost	total loop cost
i loop		
j loop		

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j?

```
for (j = 1; j < N-1; j++)  
  for (i = 0; i < N; i++)  
    a[j][i] =  $\underbrace{b[i][j-1] + b[i][j] + b[i][j+1]}_N + c[j][i];$ 
```

$\underbrace{a[j][i]}_{N/L}$ $\underbrace{b[i][j-1] + b[i][j] + b[i][j+1]}_N$ $\underbrace{c[j][i]}_{N/L}$

innermost	inner loop cost	total loop cost
i loop		
j loop		

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j?

```
for (j = 1; j < N-1; j++)  
  for (i = 0; i < N; i++)  
    a[j][i] =  $\underbrace{b[i][j-1] + b[i][j] + b[i][j+1]}_N + c[j][i];$ 
```

Annotations: N/L under $a[j][i]$, N/L under $c[j][i]$, and N under the bracketed expression.

innermost	inner loop cost	total loop cost
i loop	$N + 2N/L$	$N^2 + 2N^2/L$
j loop		

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j?

```
for (i = 0; i < N; i++)  
  for (j = 1; j < N-1; j++)  
    a[j][i] = b[i][j-1] + b[i][j] + b[i][j+1] + c[j][i];
```

innermost	inner loop cost	total loop cost
i loop	$N + 2N/L$	$N^2 + 2N^2/L$
j loop		

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j?

```
for (i = 0; i < N; i++)  
  for (j = 1; j < N-1; j++)  
    a[j][i] =  $\underbrace{b[i][j-1] + b[i][j] + b[i][j+1]}_{N/L} + c[j][i];$ 
```

Diagram illustrating the loop cost model. The code snippet shows a nested loop structure. The inner loop (j) is highlighted in orange. The cost of the inner loop is labeled as N/L , and the cost of the outer loop is labeled as N .

innermost	inner loop cost	total loop cost
i loop	$N + 2N/L$	$N^2 + 2N^2/L$
j loop		

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j?

```
for (i = 0; i < N; i++)  
  for (j = 1; j < N-1; j++)  
    a[j][i] =  $\underbrace{b[i][j-1] + b[i][j] + b[i][j+1]}_{N/L} + c[j][i];$ 
```

Diagram illustrating the loop cost model for the given code snippet. The innermost loop is the j loop, and the outer loop is the i loop. The cost of the innermost loop is N/L , and the cost of the outer loop is N .

innermost	inner loop cost	total loop cost
i loop	$N + 2N/L$	$N^2 + 2N^2/L$
j loop	$N/L + 2N$	$N^2/L + 2N^2$

Loop Cost Model

Loop cost model [K. Kennedy and K. S. McKinley, ICS 1992]:

- e.g., Which loop should be innermost: i, j? **i loop**

```
#pragma omp parallel for
```

```
for (j = 1; j < N-1; j++)
```

```
    for (i = 1; i < N-1; i++)
```

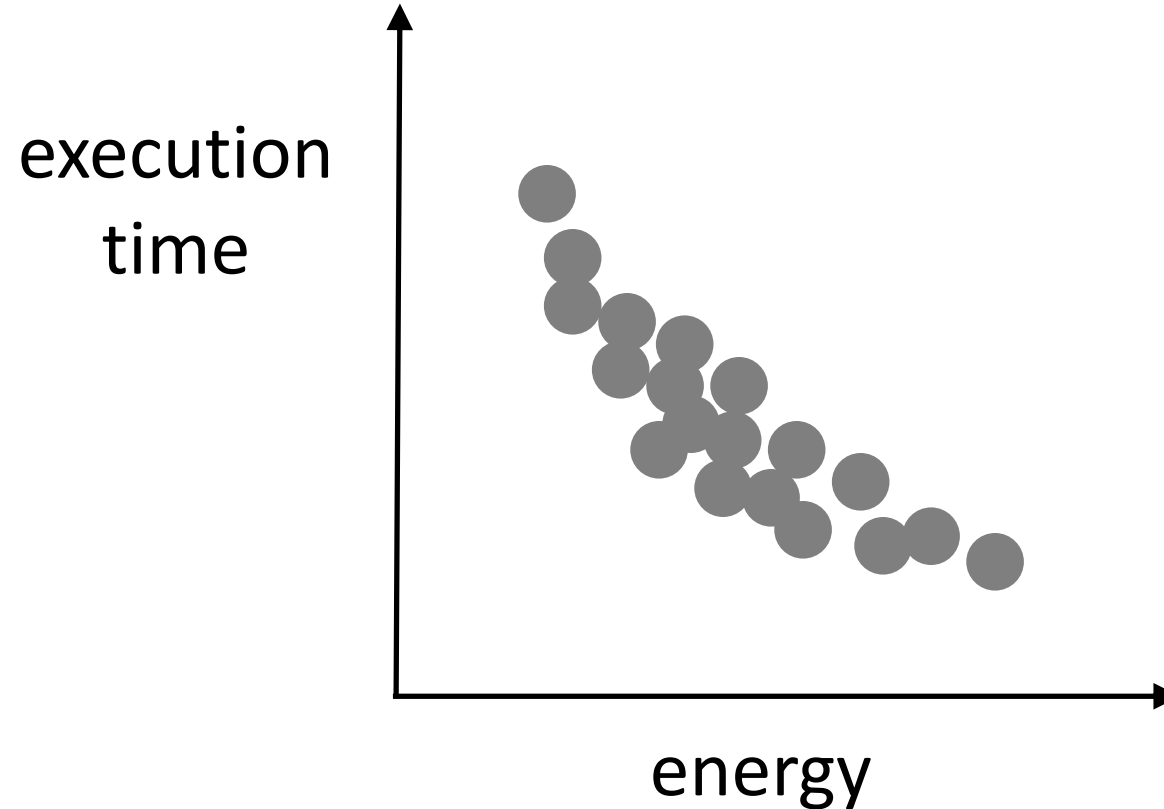
```
        a[j][i] = b[i][j-1] + b[i][j] + b[i][j+1] + c[j][i];
```

innermost	inner loop cost	total loop cost
i loop	$N + 2N/L$	$N^2 + 2N^2/L$
j loop	$N/L + 2N$	$N^2/L + 2N^2$

Approximate Computing

Not all data/computations need to be precise:

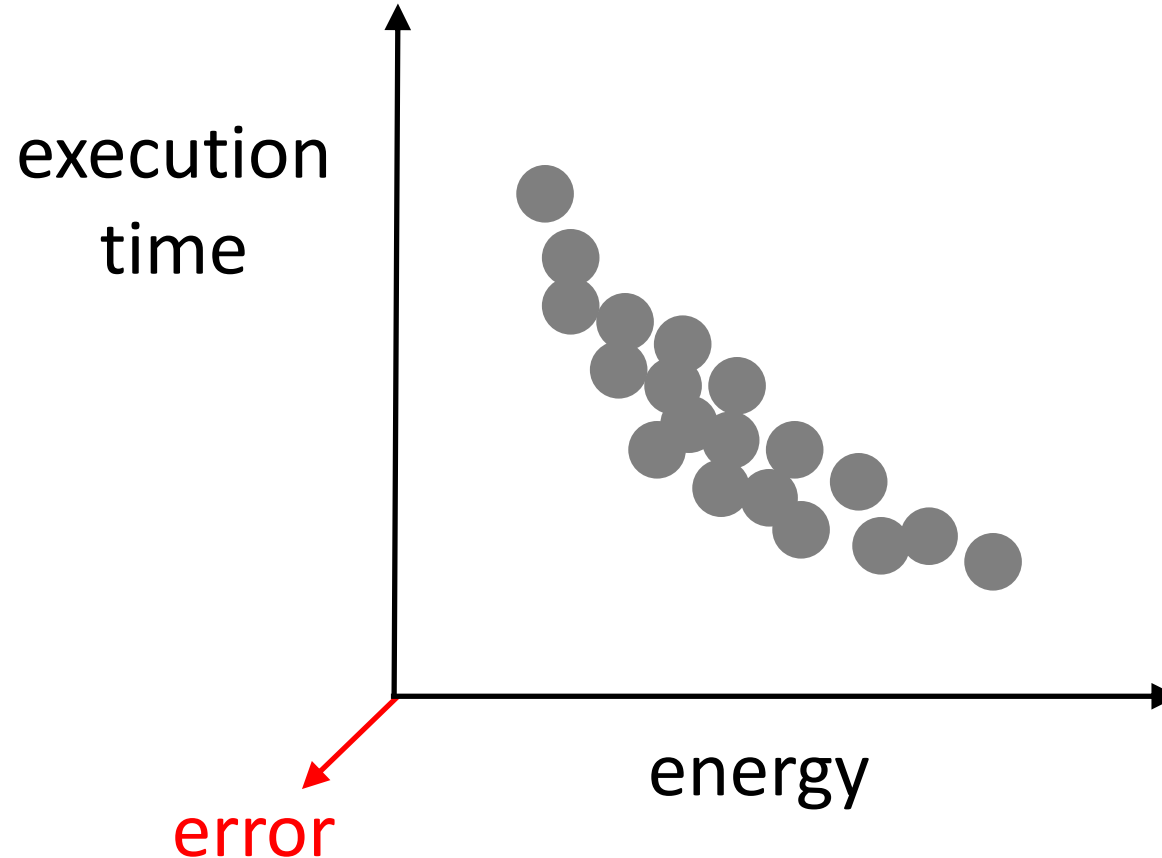
- Introduce application error (or accuracy/quality) as a design axis



Approximate Computing

Not all data/computations need to be precise:

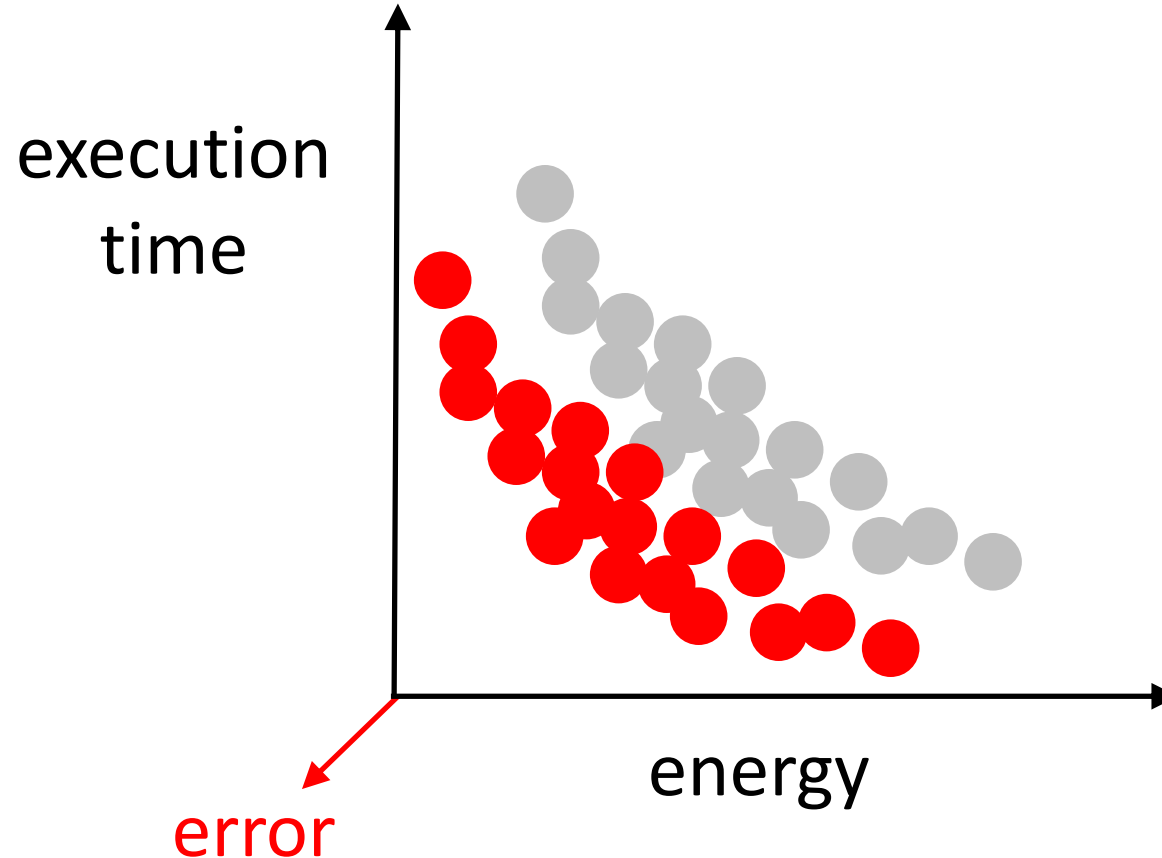
- Introduce application error (or accuracy/quality) as a design axis



Approximate Computing

Not all data/computations need to be precise:

- Introduce application error (or accuracy/quality) as a design axis



Approximate Computing

We've been approximating for years:

Approximate Computing

We've been approximating for years:

- **Fixed-point:**
 - Bounded precision in fixed-point/integer datatypes
 - e.g., What is (int) $8 / 3$?

Approximate Computing

We've been approximating for years:

- **Fixed-point:**
 - Bounded precision in fixed-point/integer datatypes
 - e.g., What is (int) $8 / 3$? **2**

Approximate Computing

We've been approximating for years:

- **Fixed-point:**
 - Bounded precision in fixed-point/integer datatypes
 - e.g., What is (int) $8 / 3$? **2**
- **Floating-point:**
 - Finite digital representation of infinitely many numbers
 - e.g., What is 1.2 in IEEE 754 floating-point?

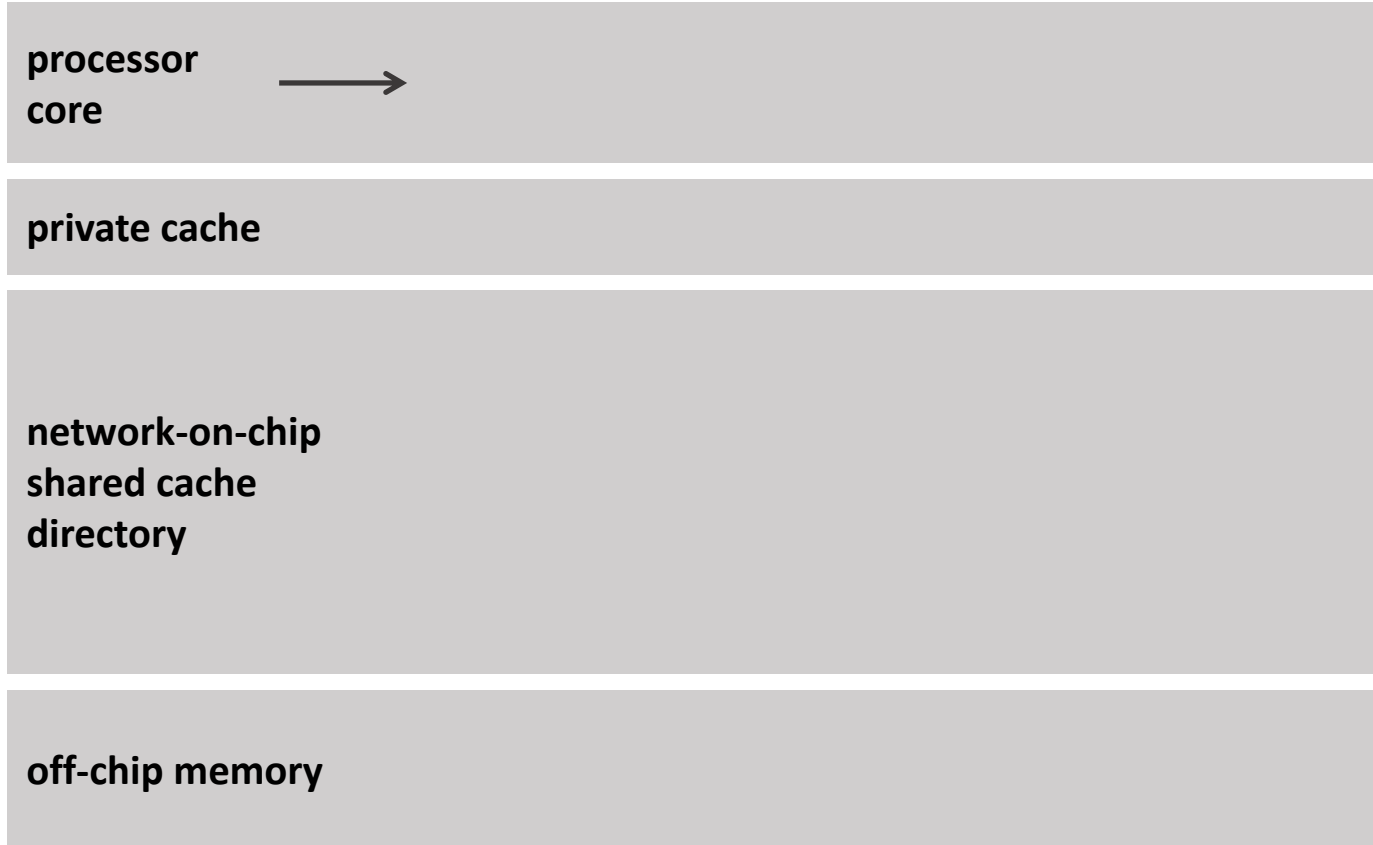
Approximate Computing

We've been approximating for years:

- **Fixed-point:**
 - Bounded precision in fixed-point/integer datatypes
 - e.g., What is (int) $8 / 3$? **2**
- **Floating-point:**
 - Finite digital representation of infinitely many numbers
 - e.g., What is 1.2 in IEEE 754 floating-point? **1.2000000476837158203125**

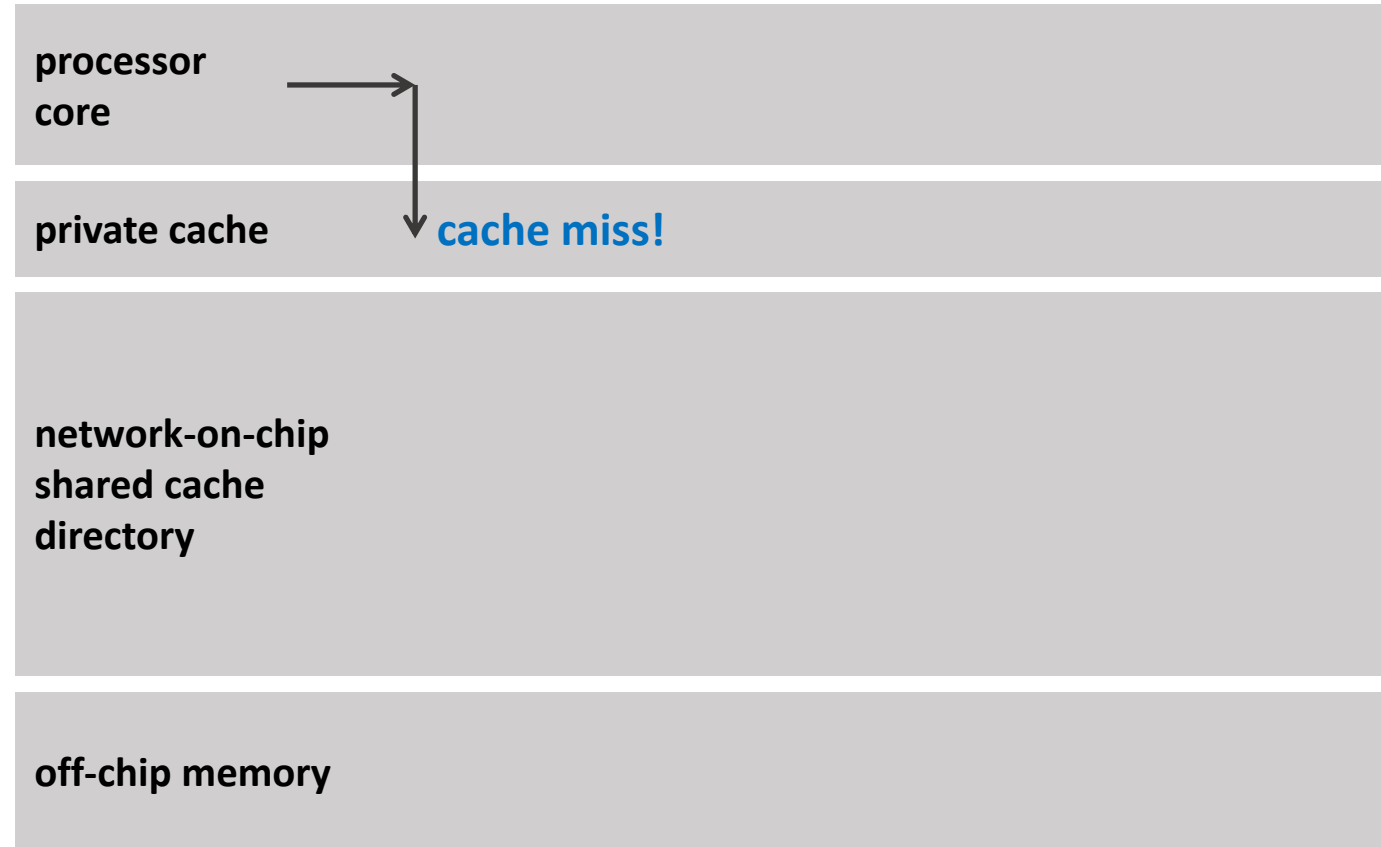
Load Value Approximation

An instruction requests a memory block



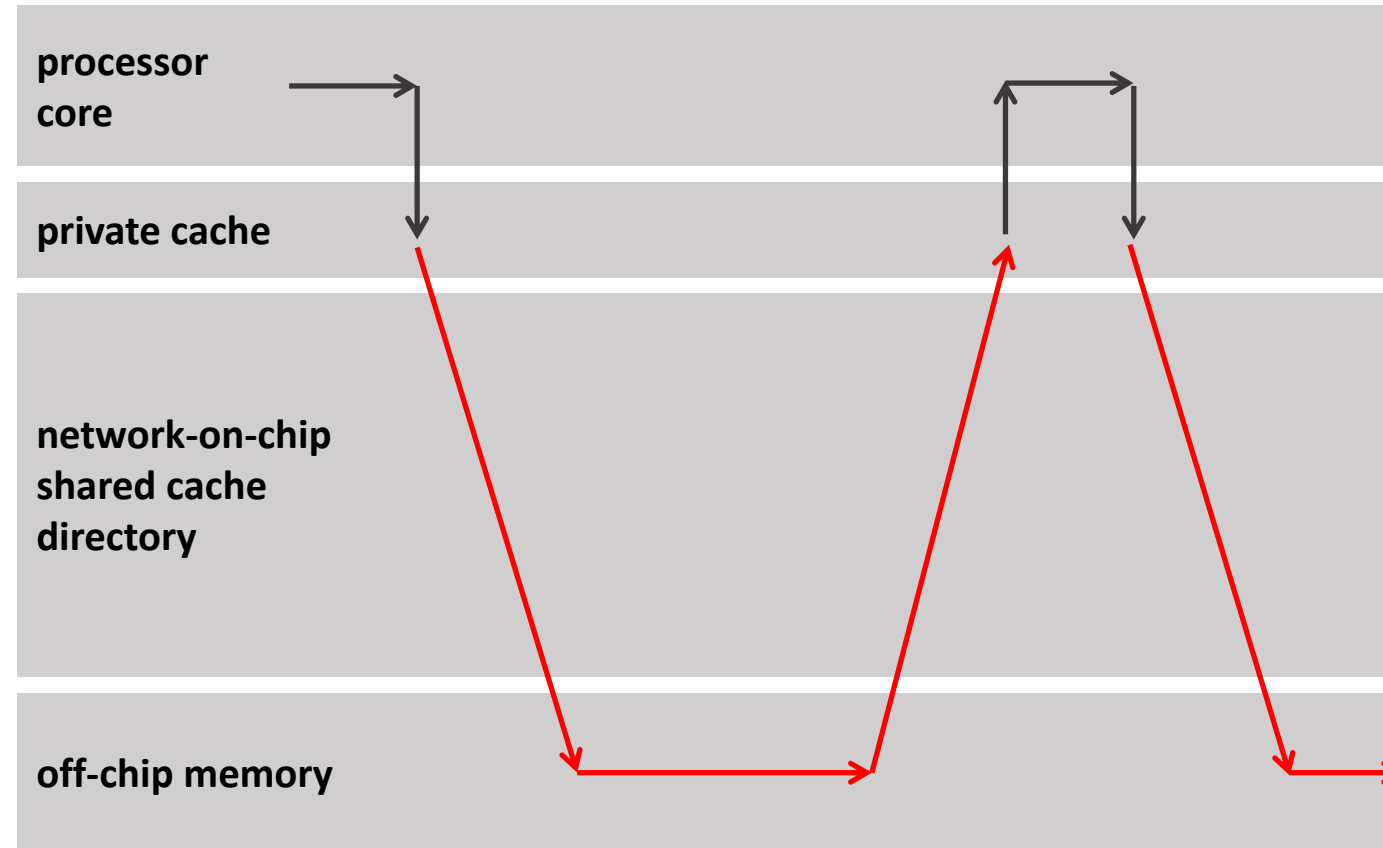
Load Value Approximation

An instruction requests a memory block



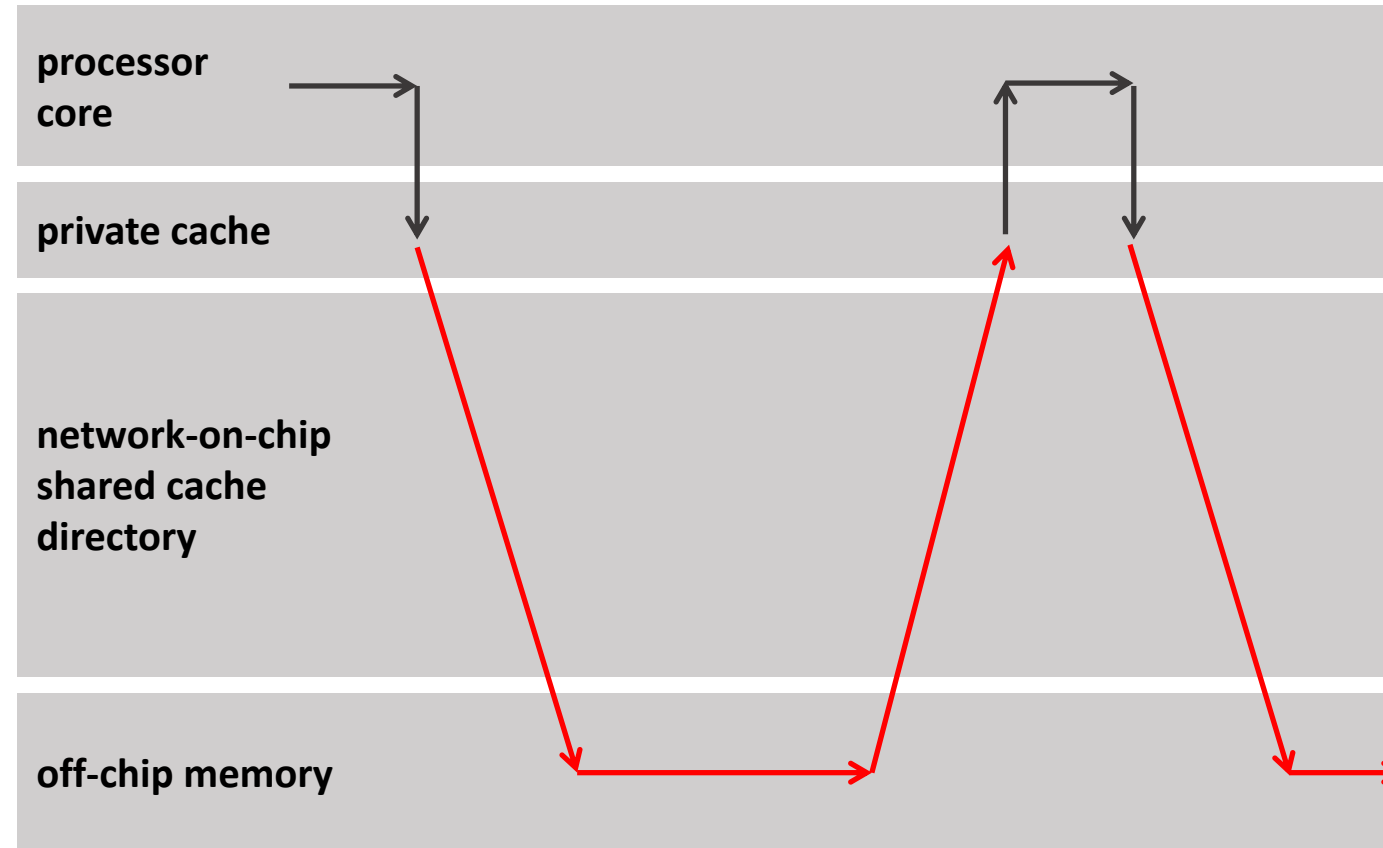
Load Value Approximation

An instruction requests a memory block



Load Value Approximation

An instruction requests *information; information is stored in a memory block*



Load Value Approximation

An instruction requests
*information; information is
stored in* a memory block

- **Approximator:** decouples the computing on data from the fetching of data

processor
core

private cache

approximator

network-on-chip
shared cache
directory

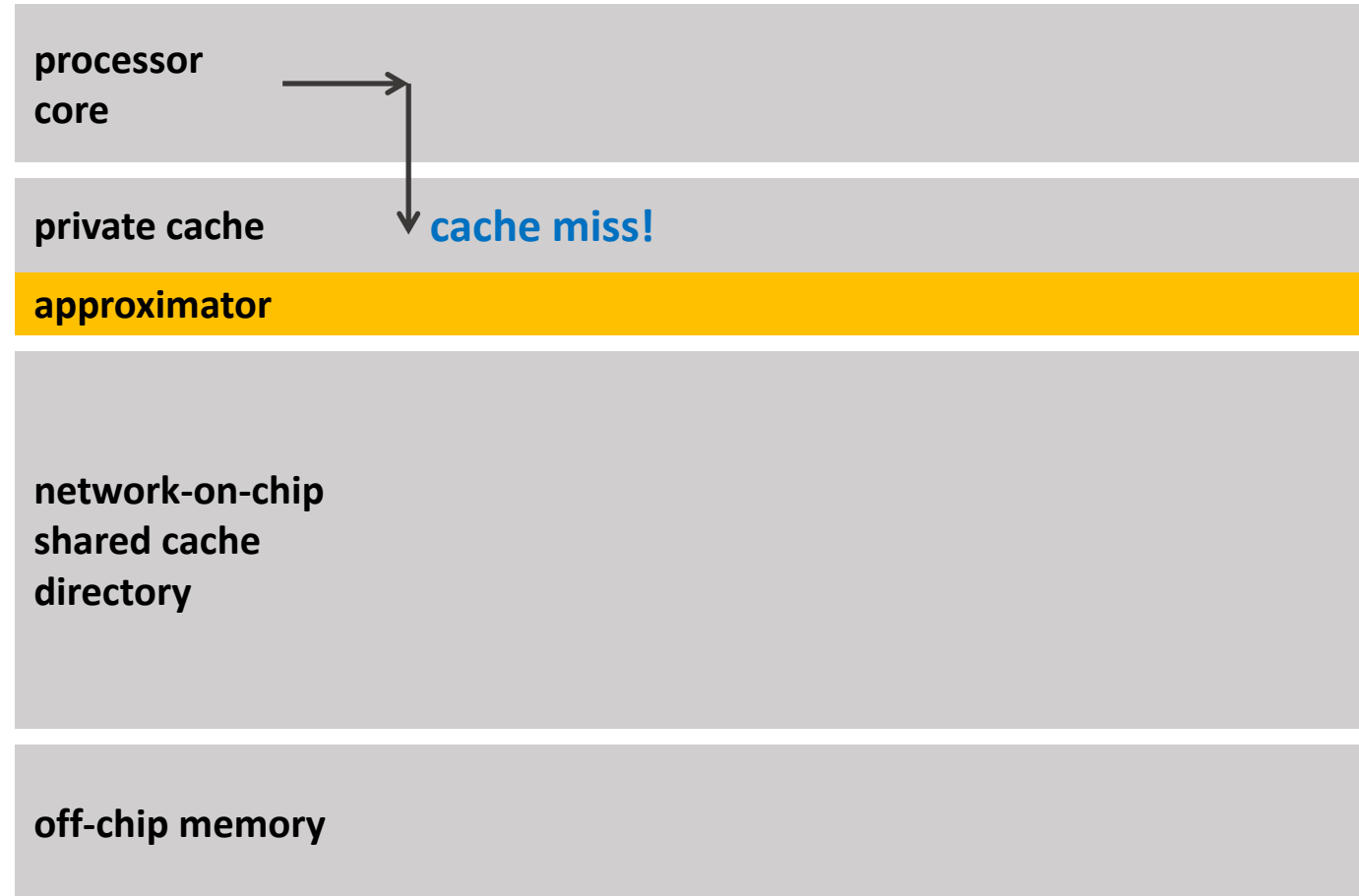
off-chip memory



Load Value Approximation

An instruction requests *information; information is stored in* a memory block

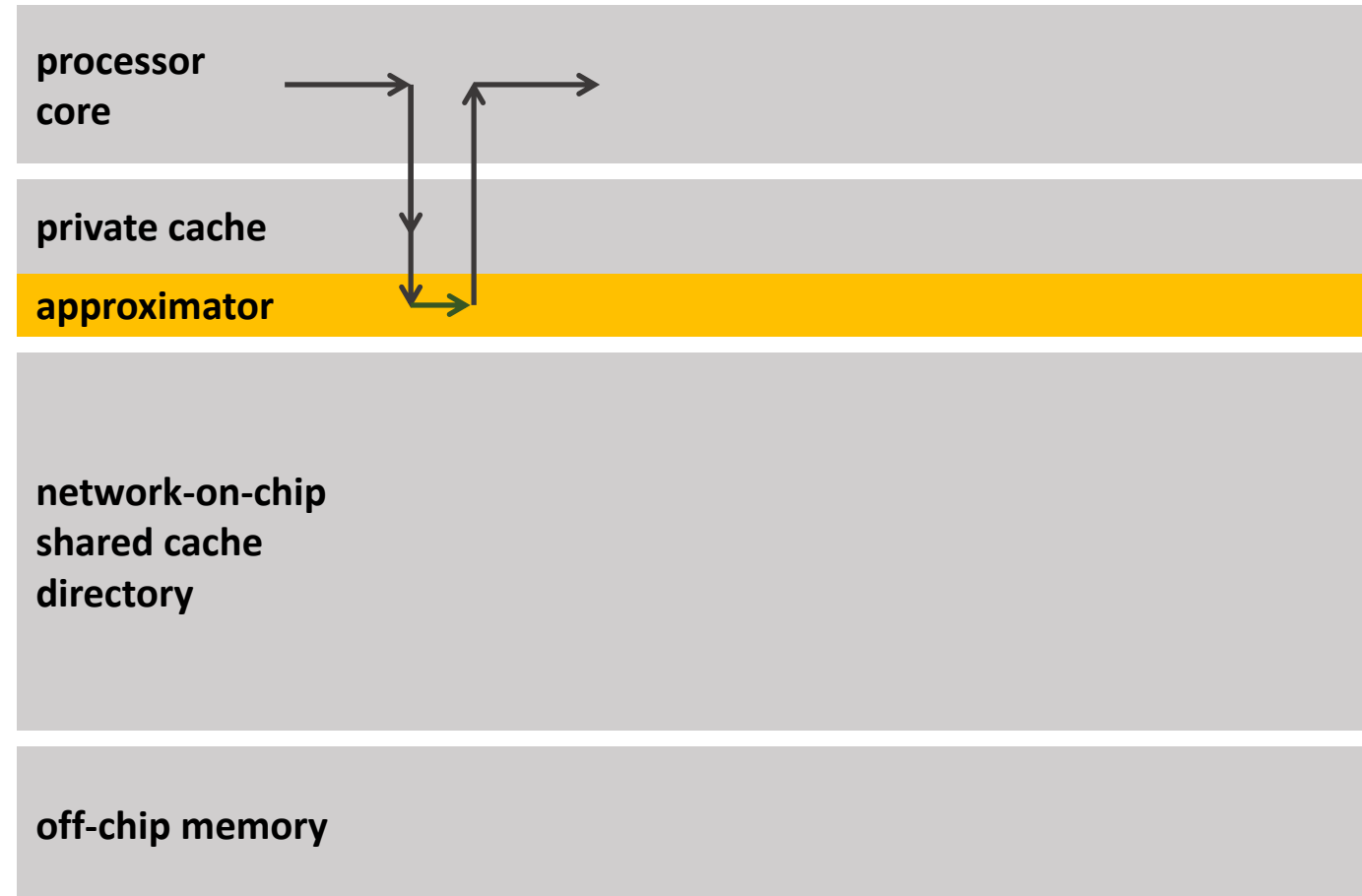
- **Approximator:** decouples the computing on data from the fetching of data



Load Value Approximation

An instruction requests *information; information is stored in* a memory block

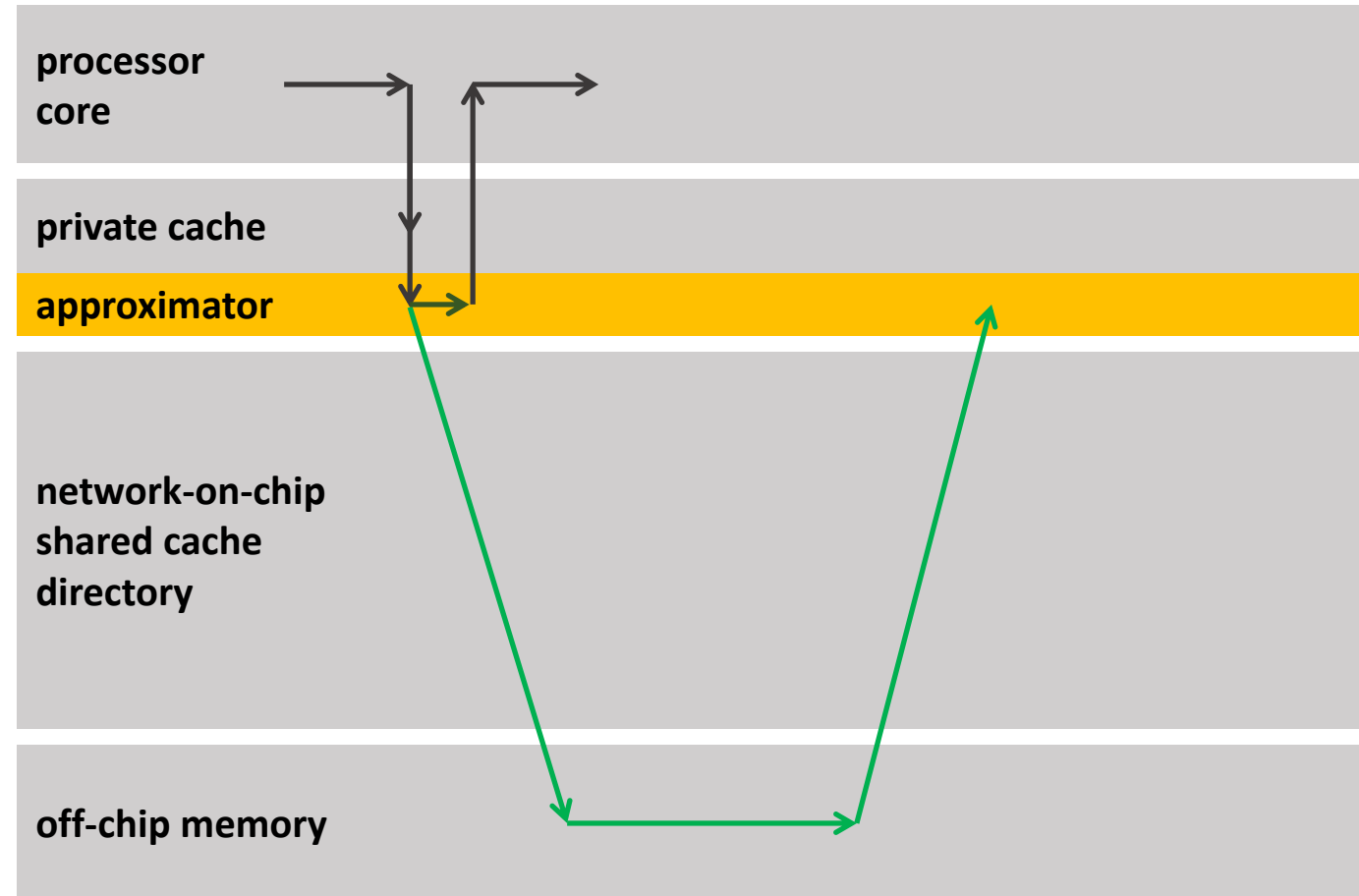
- **Approximator:** decouples the computing on data from the fetching of data



Load Value Approximation

An instruction requests *information; information is stored in a memory block*

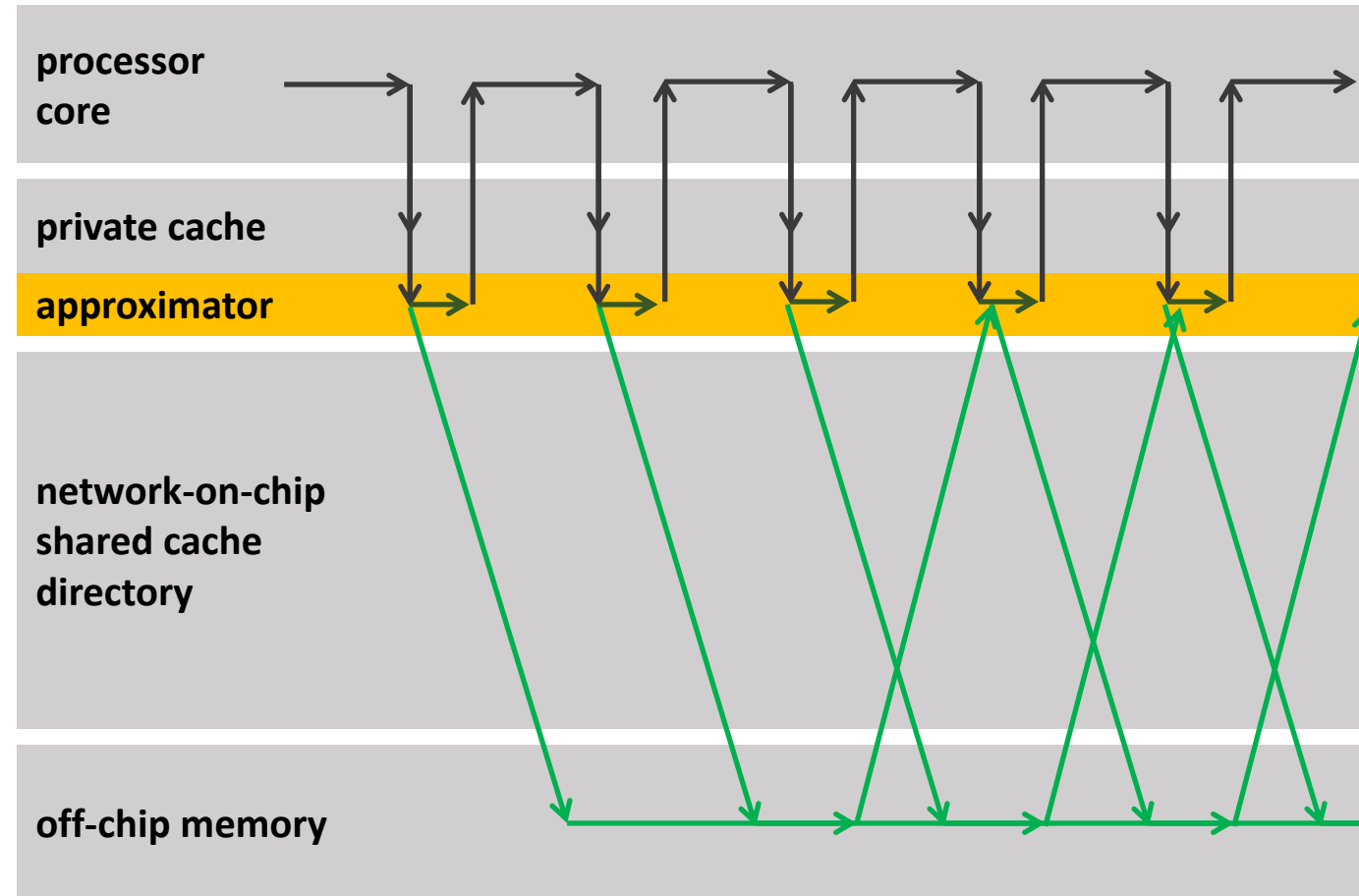
- **Approximator:** decouples the computing on data from the fetching of data
 - + Takes miss latency off the critical path; no rollbacks



Load Value Approximation

An instruction requests *information; information is stored in a memory block*

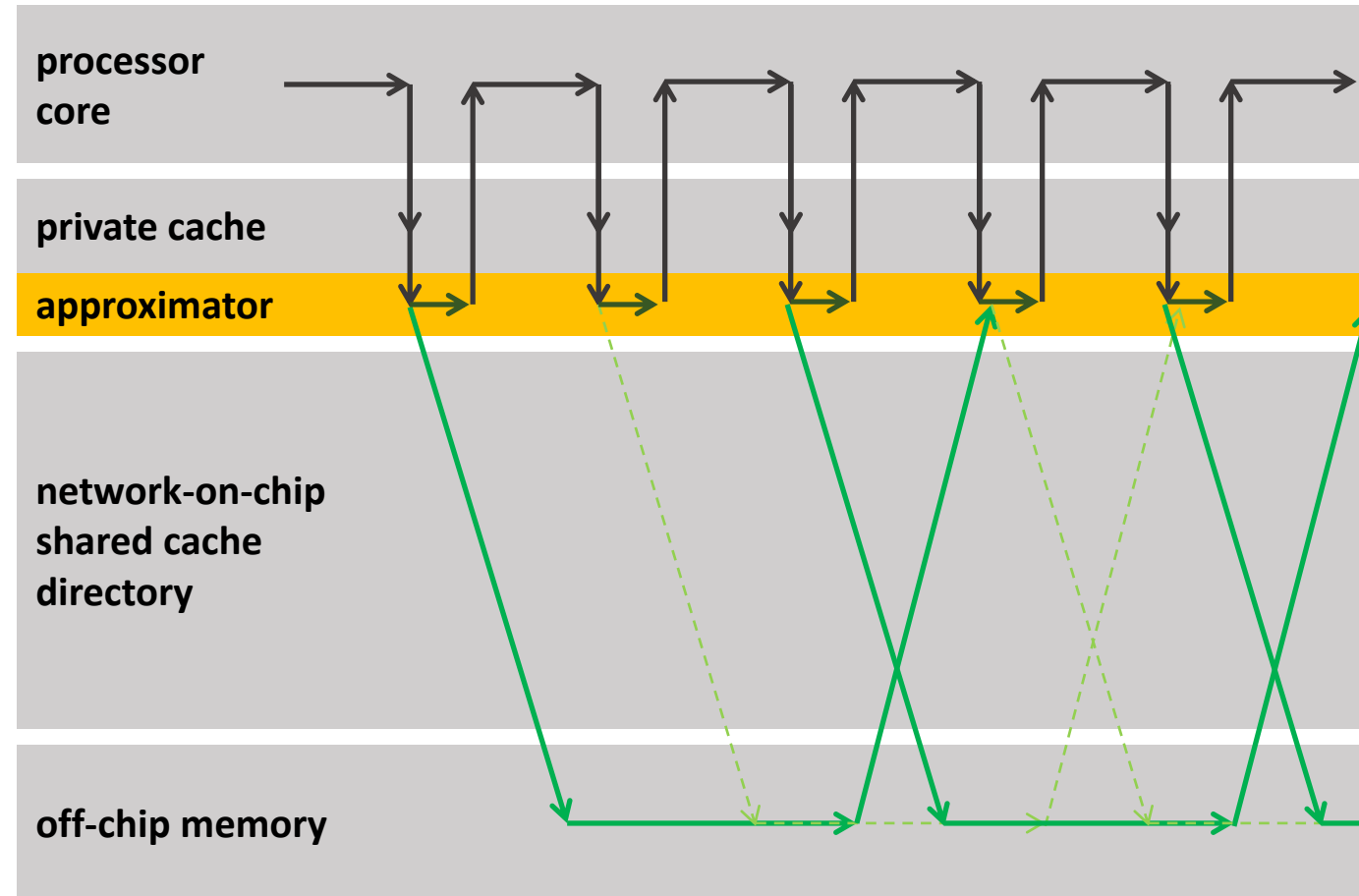
- **Approximator:** decouples the computing on data from the fetching of data
 - + Takes miss latency off the critical path; no rollbacks



Load Value Approximation

An instruction requests *information; information is stored in a memory block*

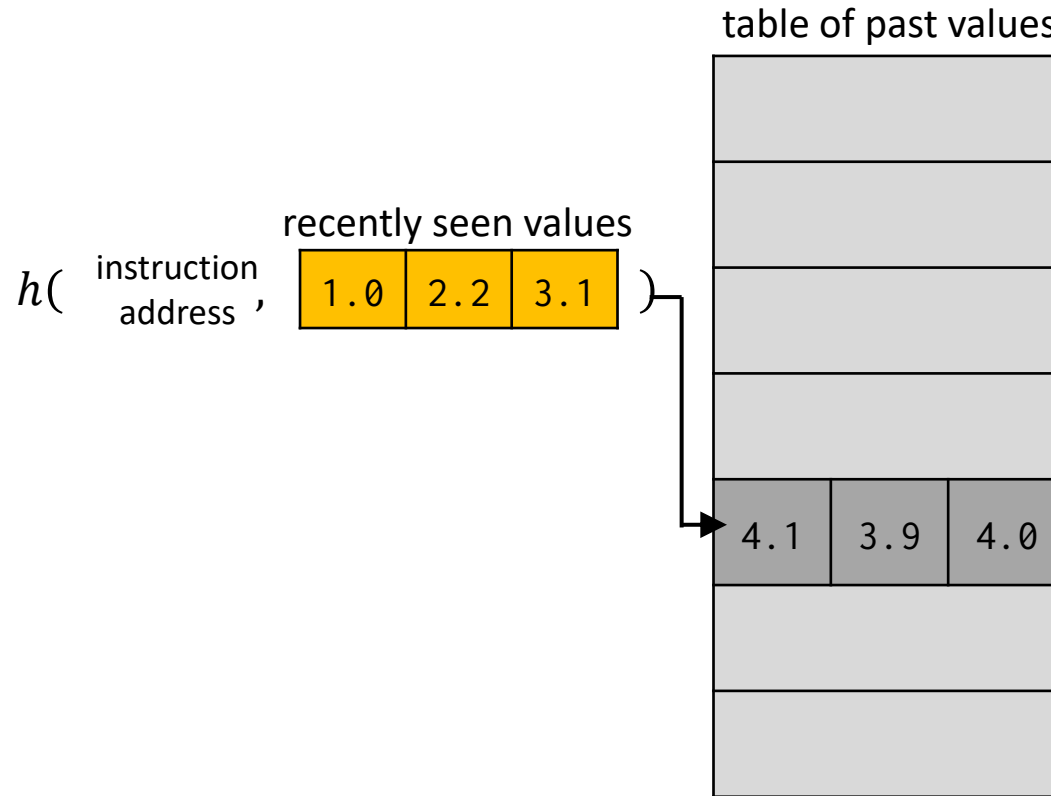
- **Approximator:** decouples the computing on data from the fetching of data
 - + Takes miss latency off the critical path; no rollbacks
 - + When accurate enough, don't fetch as often and save energy
- **Approximation degree:** number of misses served per fetched block



Load Value Approximation

Leverage **approximate value locality** for simple approximators:

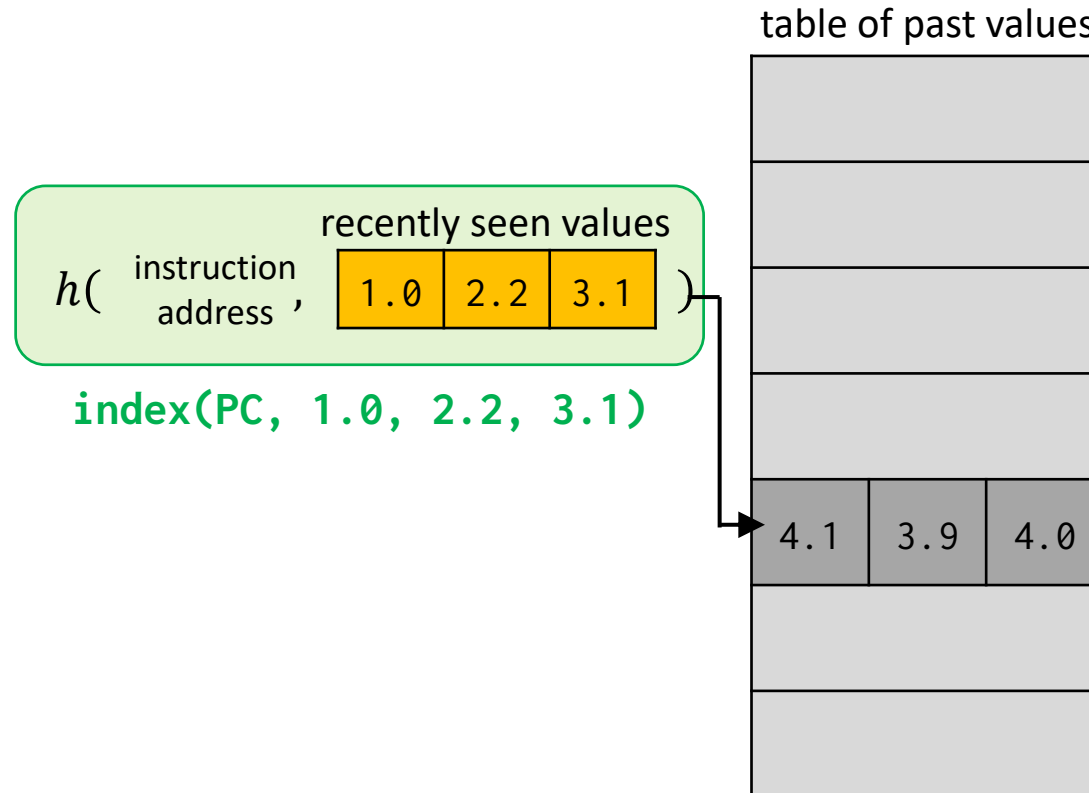
- Many data values are similar to or can be approximated from prior values



Load Value Approximation

Leverage **approximate value locality** for simple approximators:

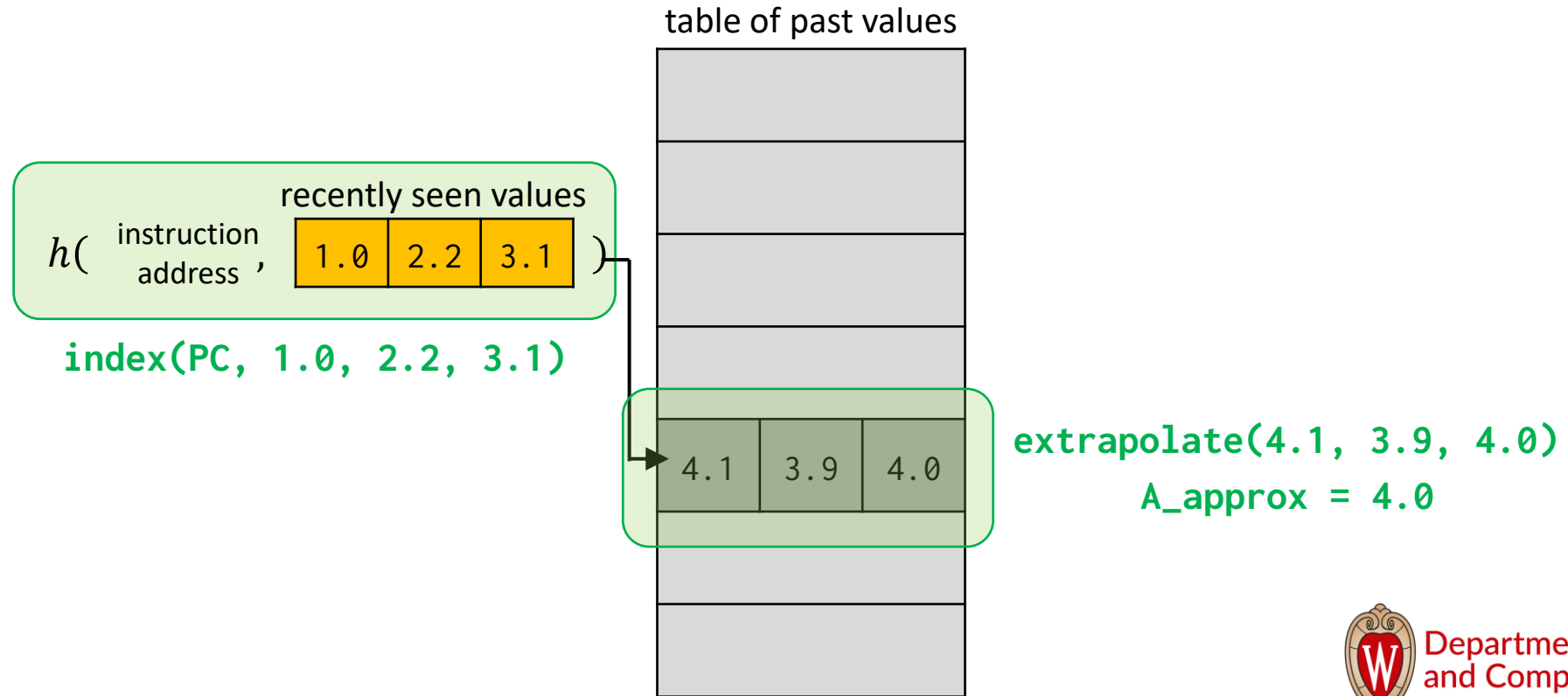
- Many data values are similar to or can be approximated from prior values



Load Value Approximation

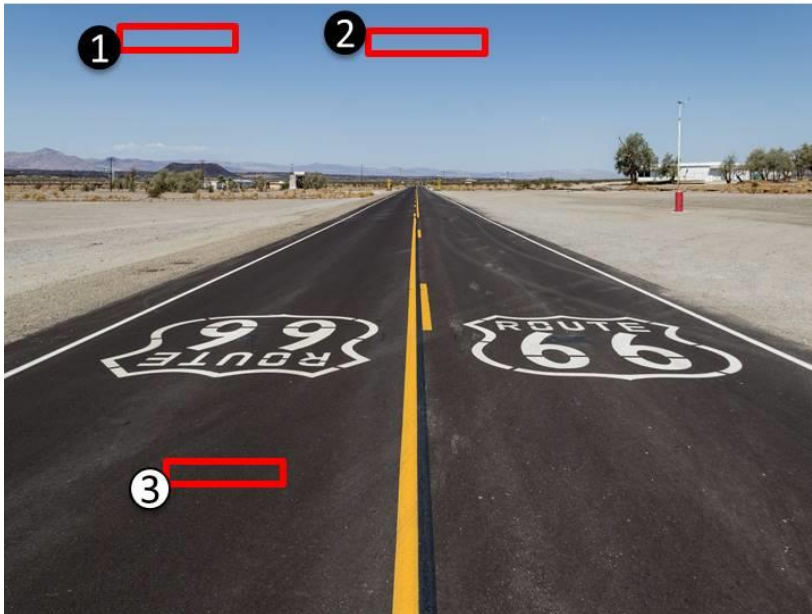
Leverage **approximate value locality** for simple approximators:

- Many data values are similar to or can be approximated from prior values
 - Can use reduced-precision arithmetic for index and extrapolation functions



Approximate Value Similarity

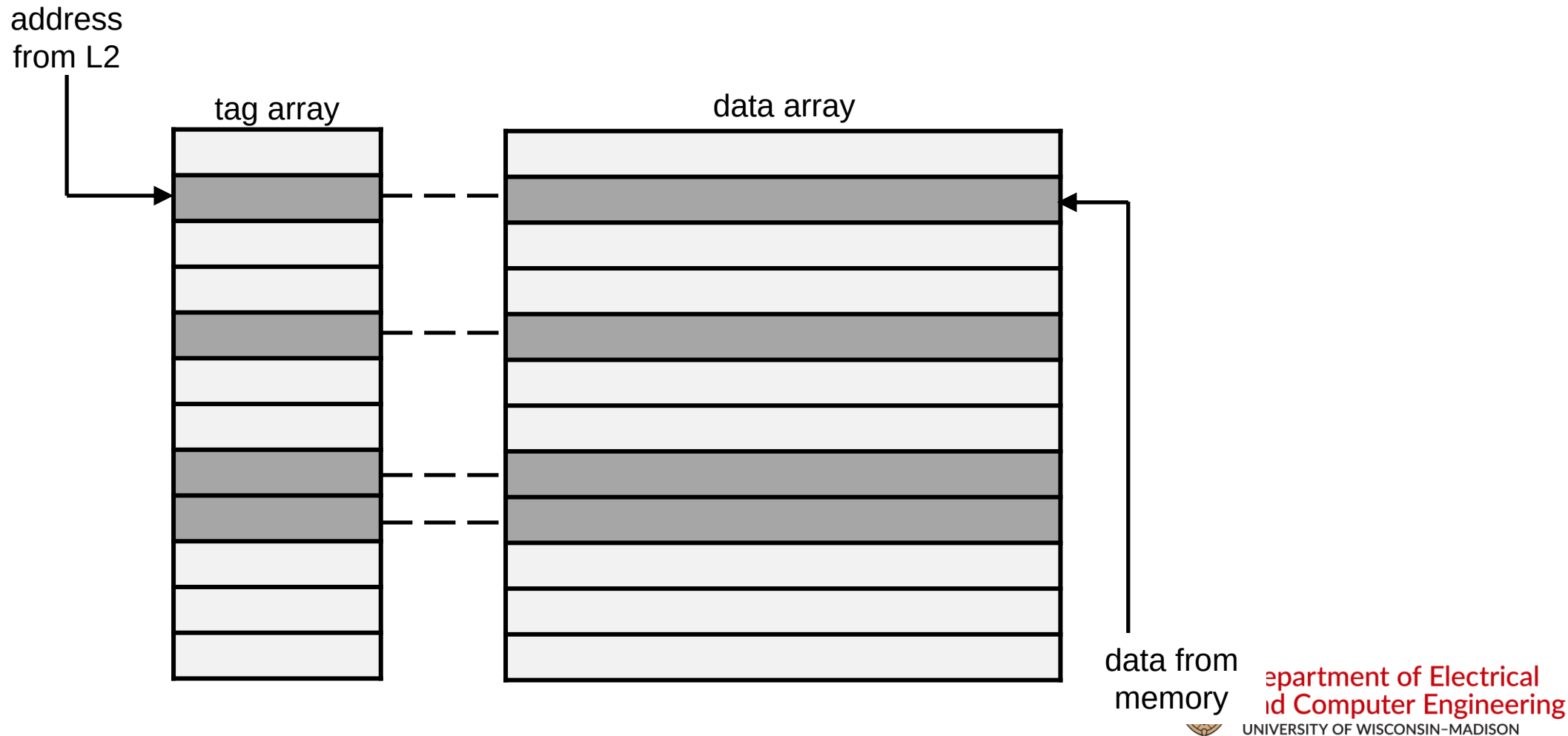
Two data blocks are **approximately similar** (*doppelgängers*) if replacing the values of one with the other still results in acceptable application output in the end



1	92	131	183	91	132	186	} approximately similar
2	90	131	185	93	133	184	
3	35	31	29	43	38	37	

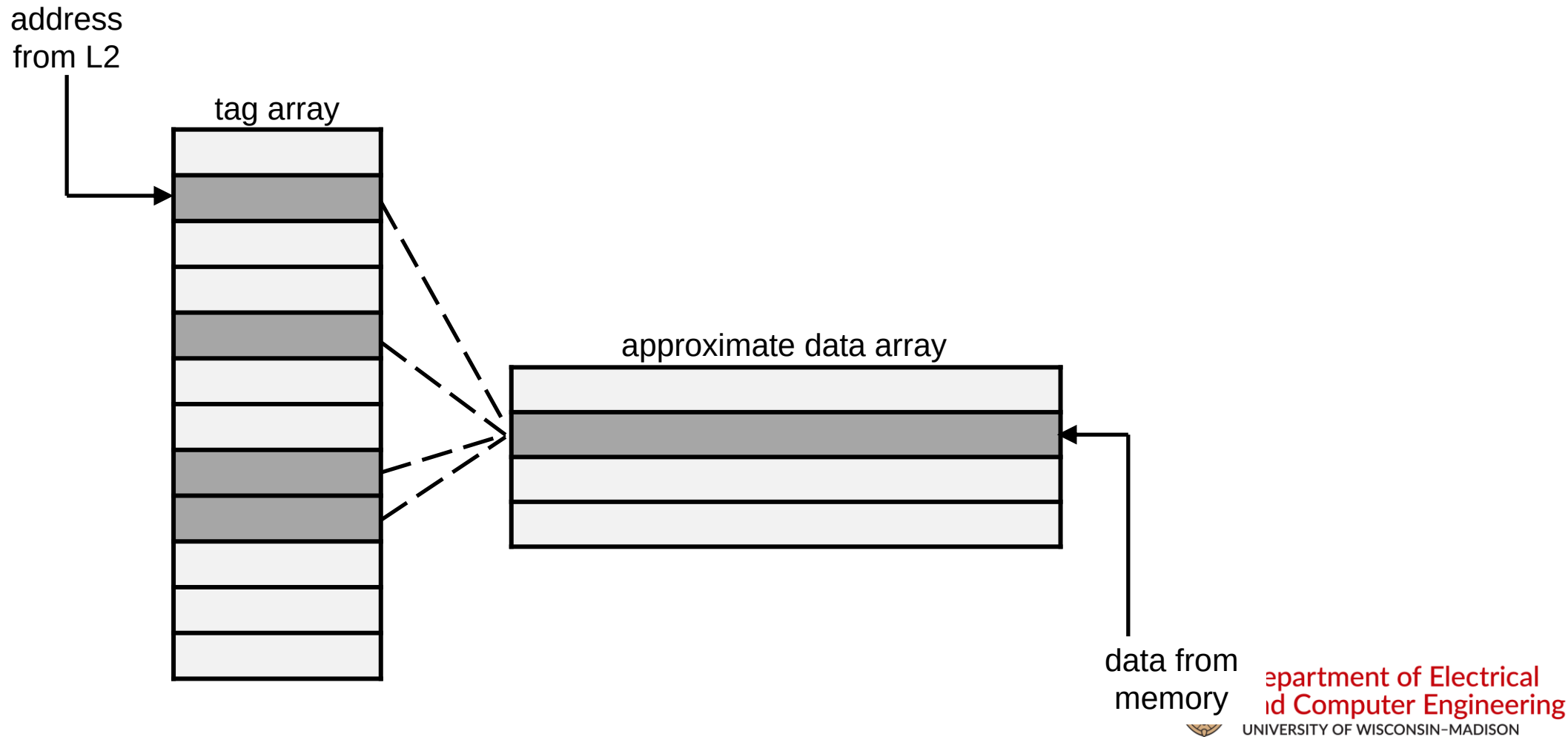
Approximate Value Similarity

In conventional cache, often have multiple copies of similar data



Approximate Value Similarity

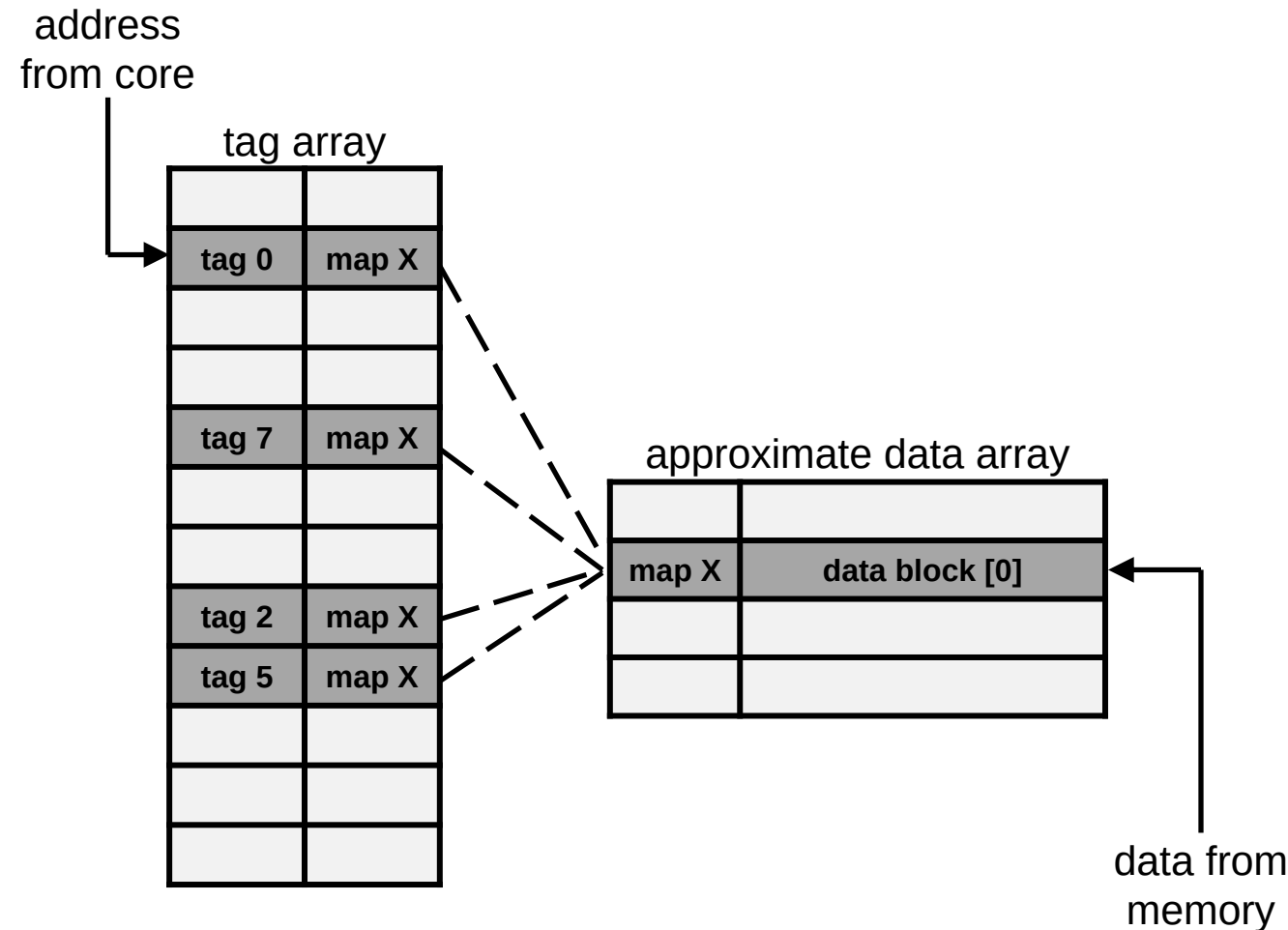
In conventional cache, often have multiple copies of similar data



Approximate Value Similarity

Doppelgänger Cache: Treat data array as a *value-indexed, value-tagged* cache:

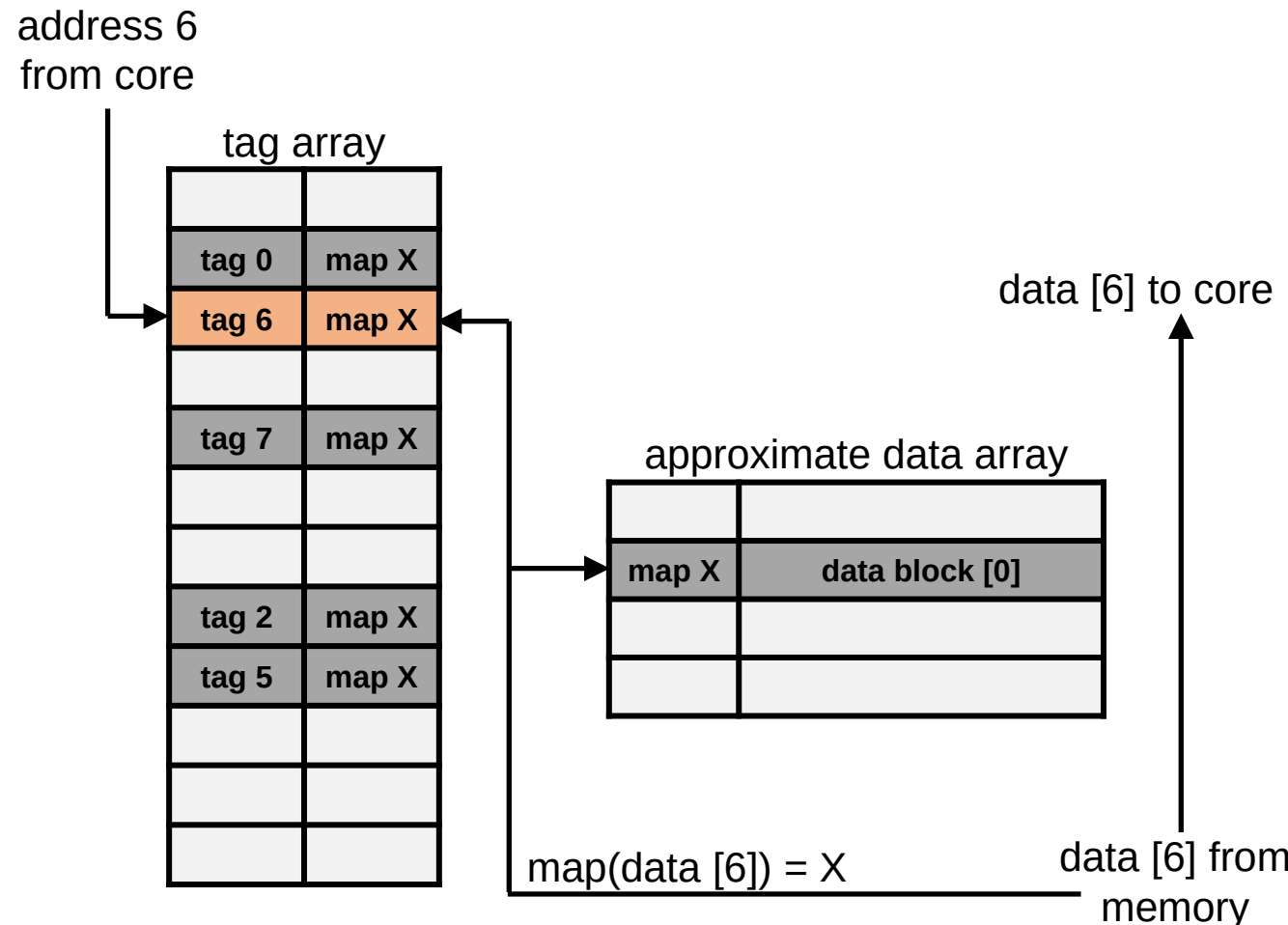
- Use **map** value to represent the *likeness* of a block
 - Blocks at addrs A and B are similar if $\text{map}([A]) = \text{map}([B])$



Approximate Value Similarity

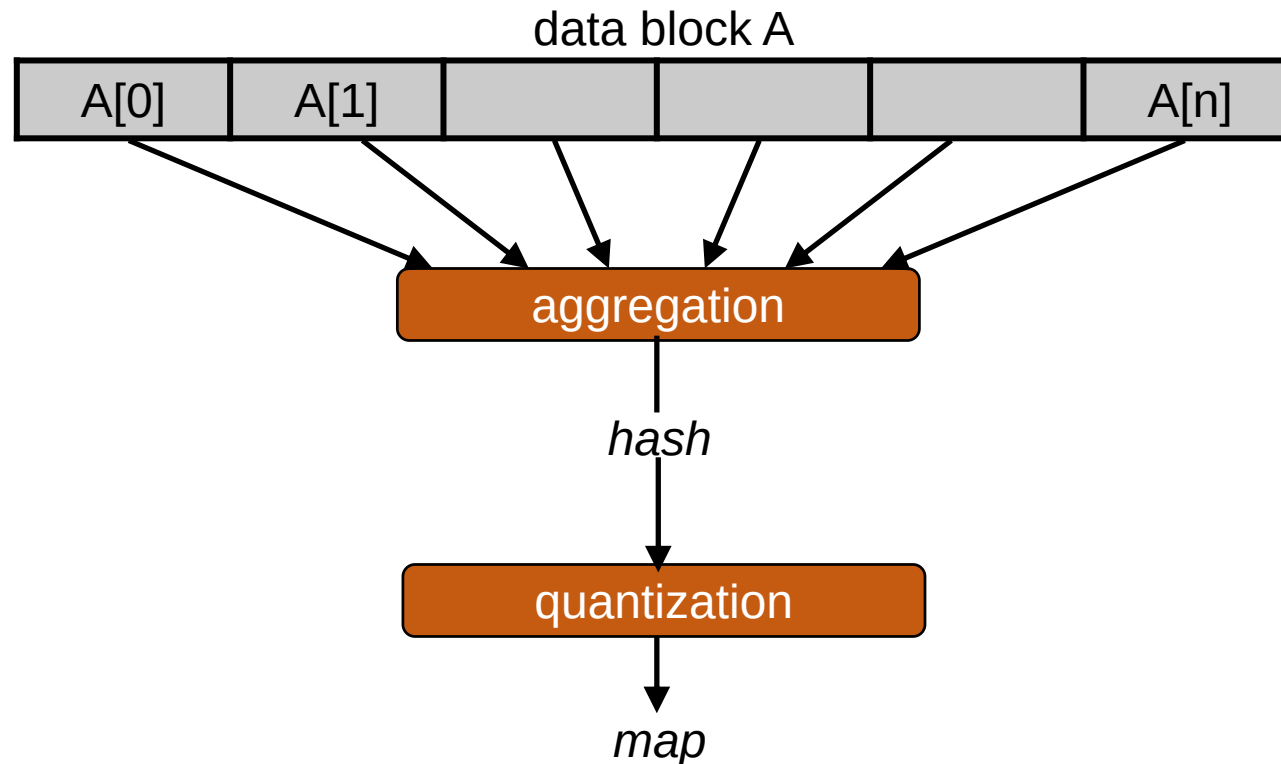
Doppelgänger Cache: Treat data array as a *value-indexed, value-tagged* cache:

- Use **map** value to represent the *likeness* of a block
 - Blocks at addrs A and B are similar if $\text{map}([A]) = \text{map}([B])$
 - Compute map value off the critical path on a miss
 - e.g., Hit in data array: the data block at addr 6 is deemed similar to the data blocks at addrs 0, 2, 5 and 7



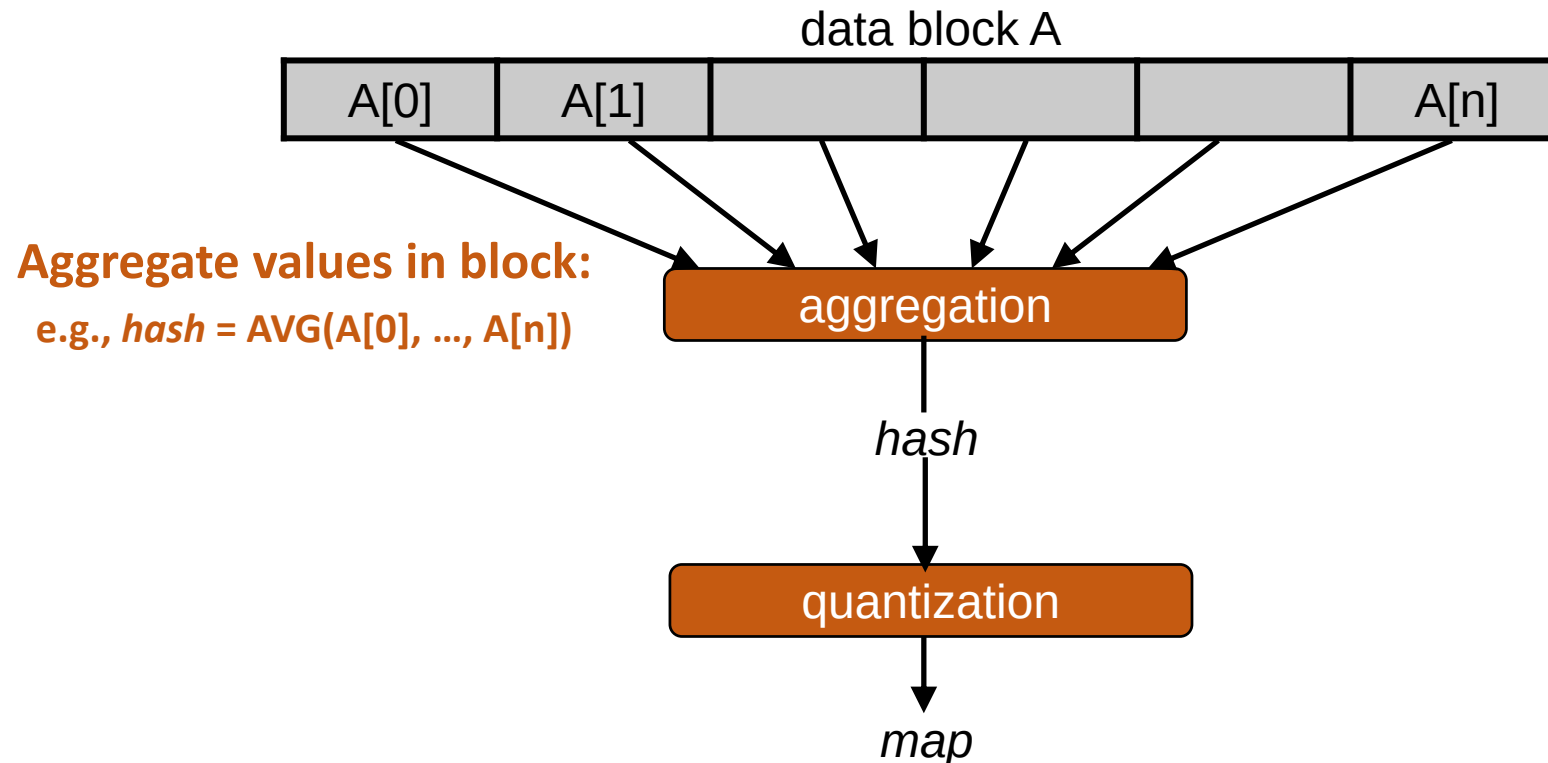
Approximate Value Similarity

Two-step process for generating map values:



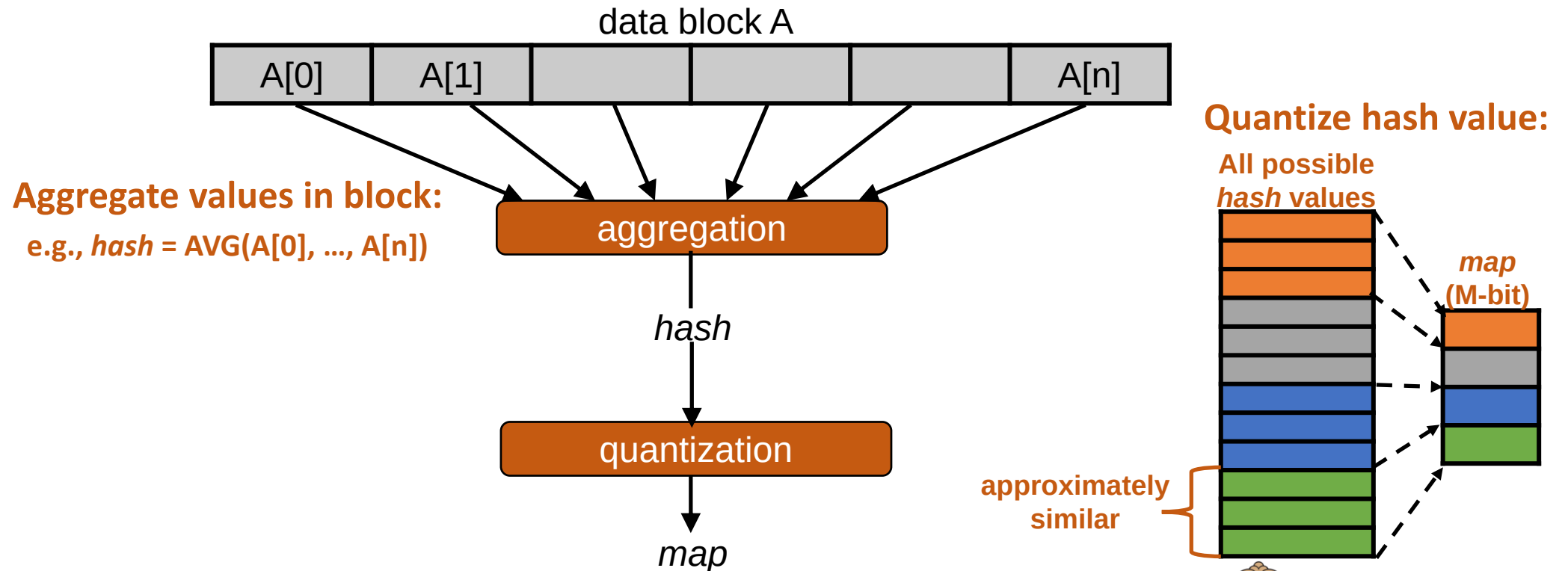
Approximate Value Similarity

Two-step process for generating map values:



Approximate Value Similarity

Two-step process for generating map values:



Approximate Spatio-Value Similarity

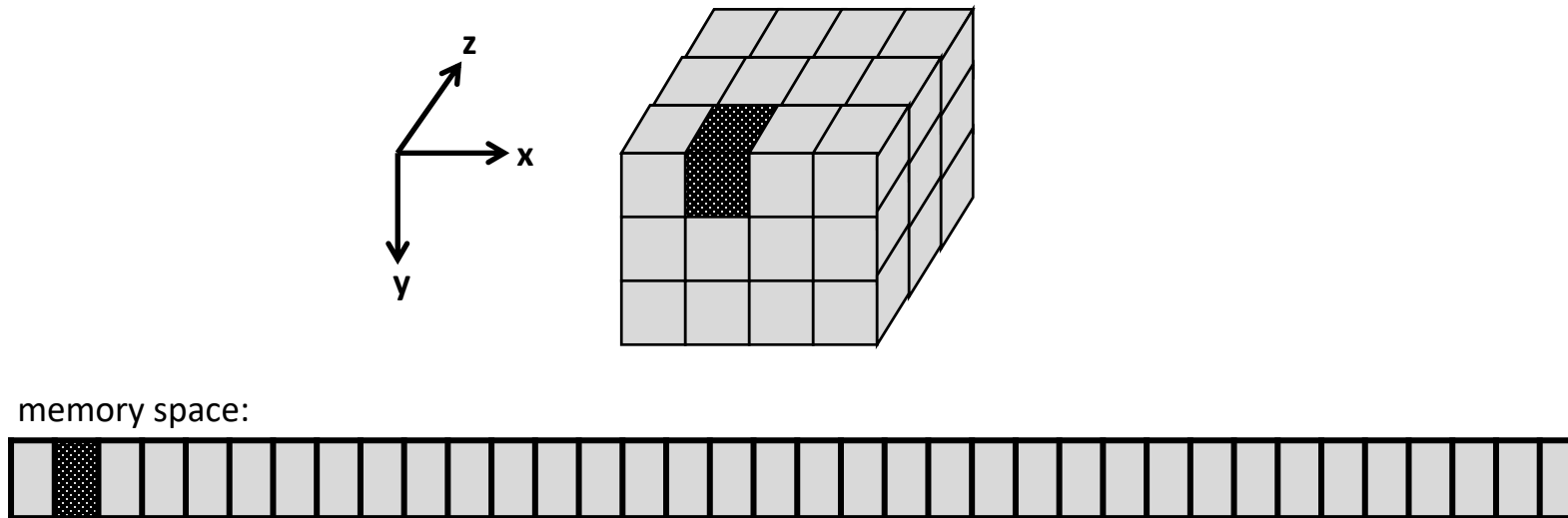
Regularity in where approximately similar values reside in memory

- Where data is located $\leftrightarrow$ What value is encoded in the data

Approximate Spatio-Value Similarity

Regularity in where approximately similar values reside in memory

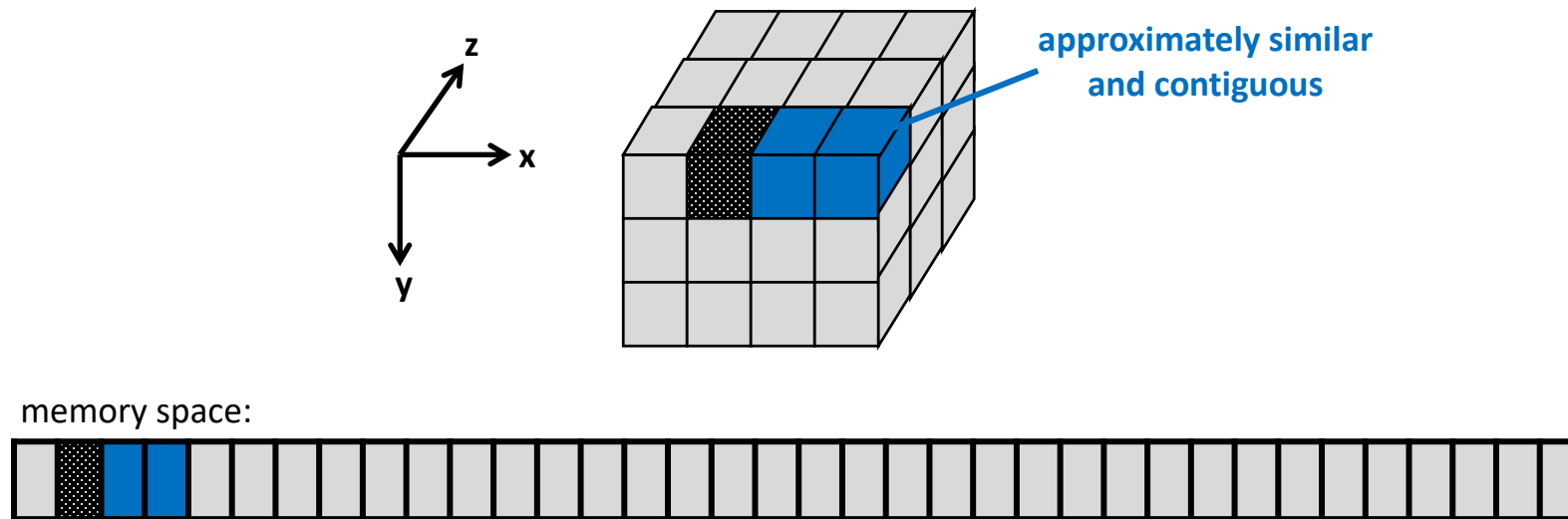
- Where data is located $\leftrightarrow$ What value is encoded in the data



Approximate Spatio-Value Similarity

Regularity in where approximately similar values reside in memory

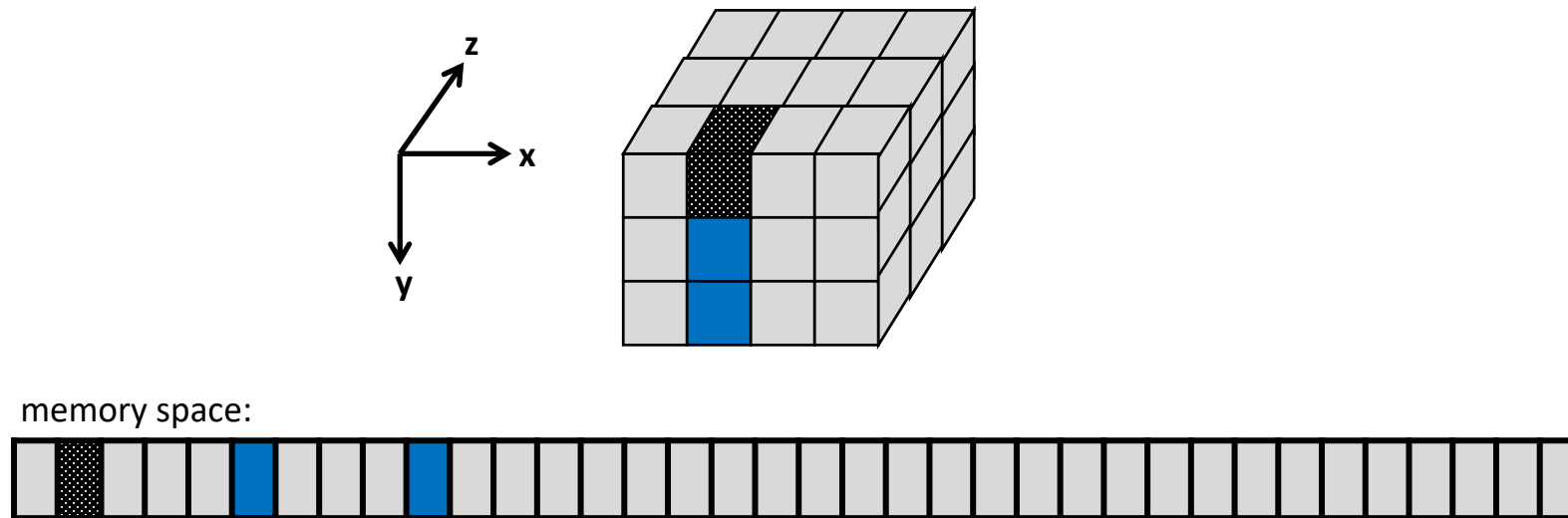
- Where data is located $\leftrightarrow$ What value is encoded in the data



Approximate Spatio-Value Similarity

Regularity in where approximately similar values reside in memory

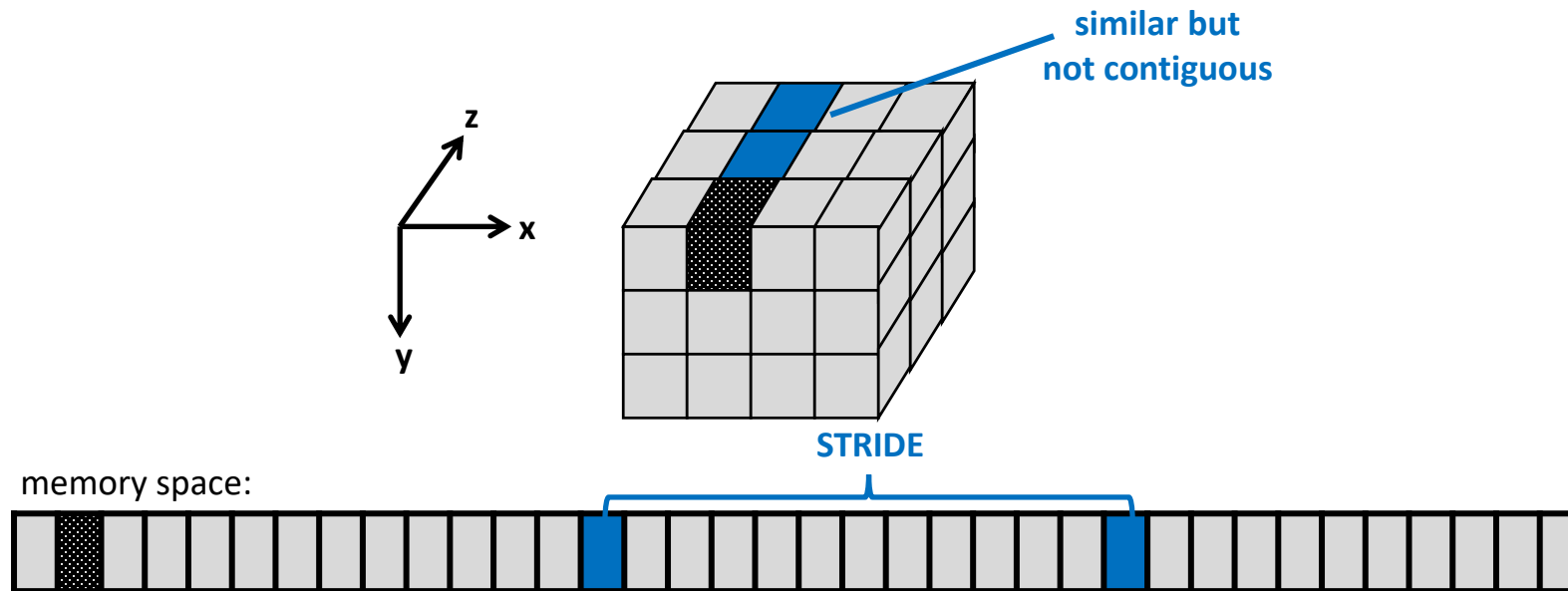
- Where data is located $\leftrightarrow$ What value is encoded in the data



Approximate Spatio-Value Similarity

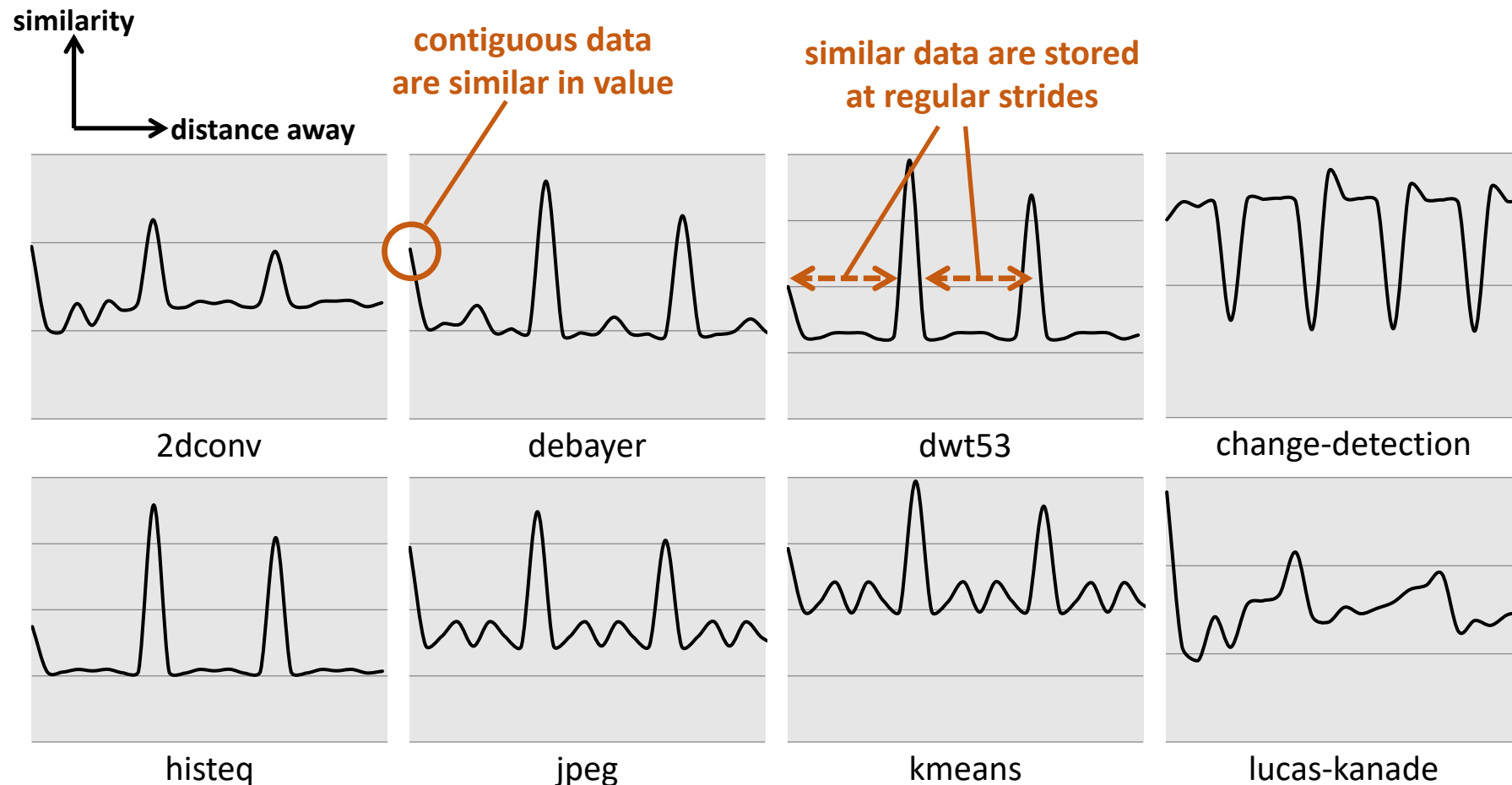
Regularity in where approximately similar values reside in memory

- Where data is located $\leftrightarrow$ What value is encoded in the data



Approximate Spatio-Value Similarity

Given any data block, how similar is it to the block that is distance X away from it?

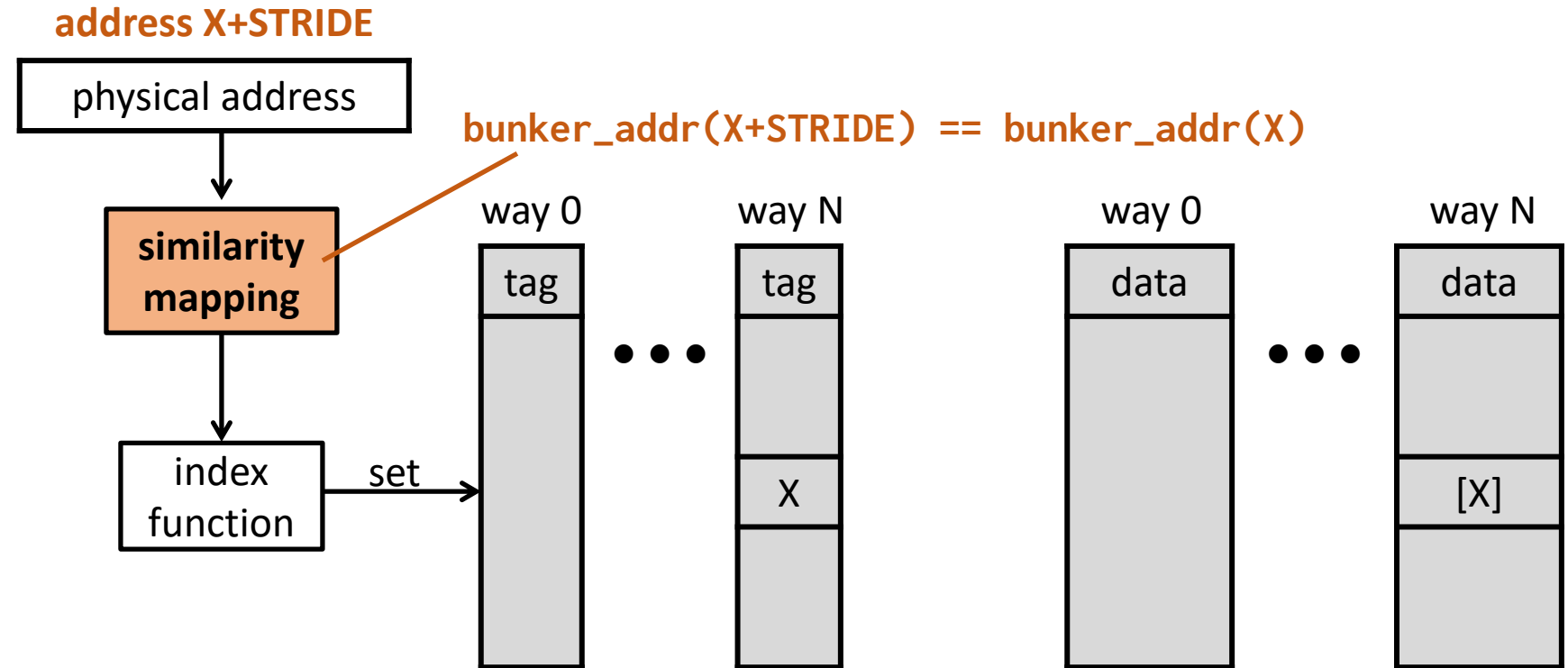


Approximate Spatio-Value Similarity

Bunker Cache:

Treat address X and address $X + \text{STRIDE}$ as if they are one and the same

- Employs **similarity mapping**: virtual address translation within the cache

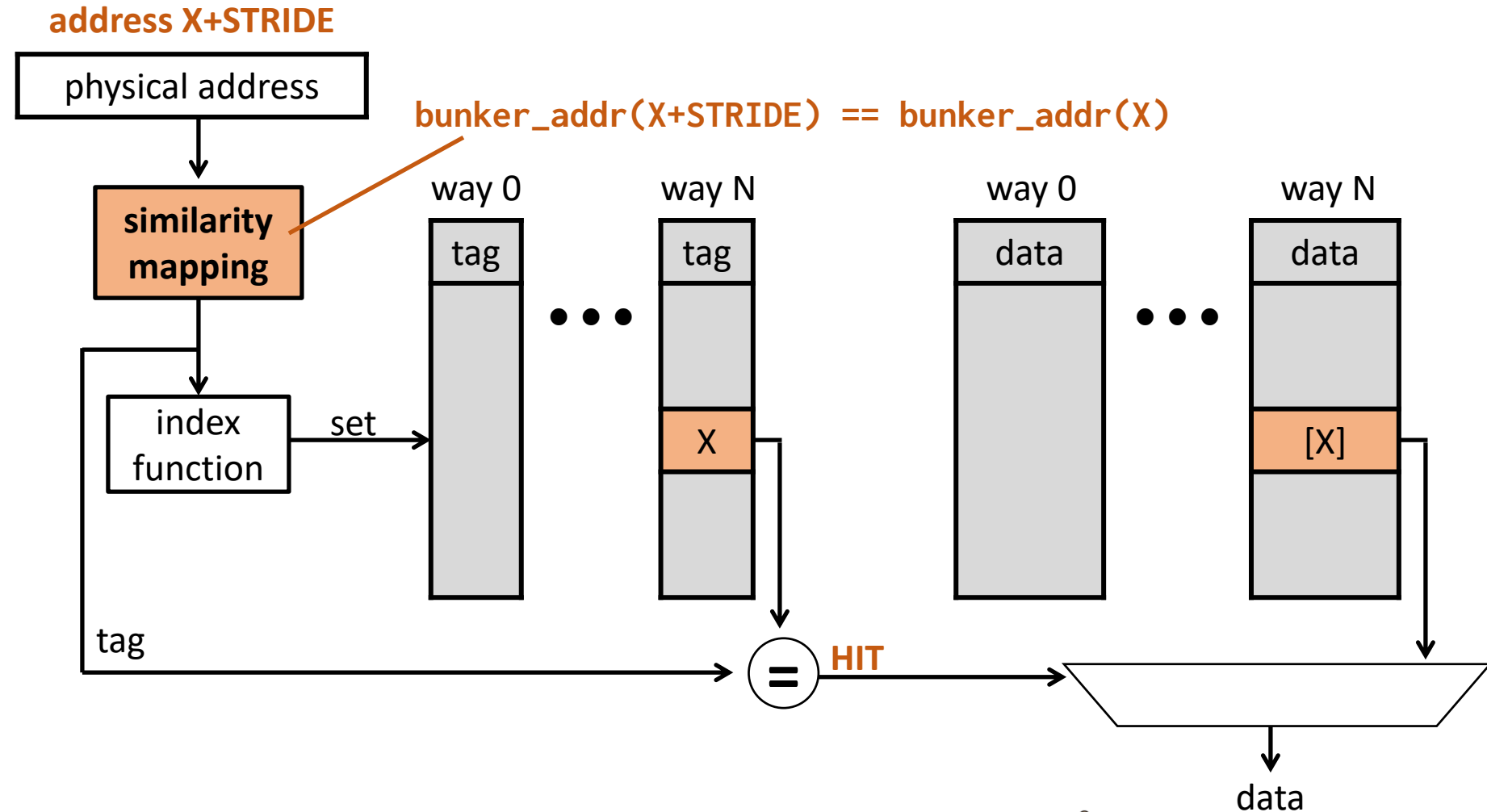


Approximate Spatio-Value Similarity

Bunker Cache:

Treat address X and address $X + \text{STRIDE}$ as if they are one and the same

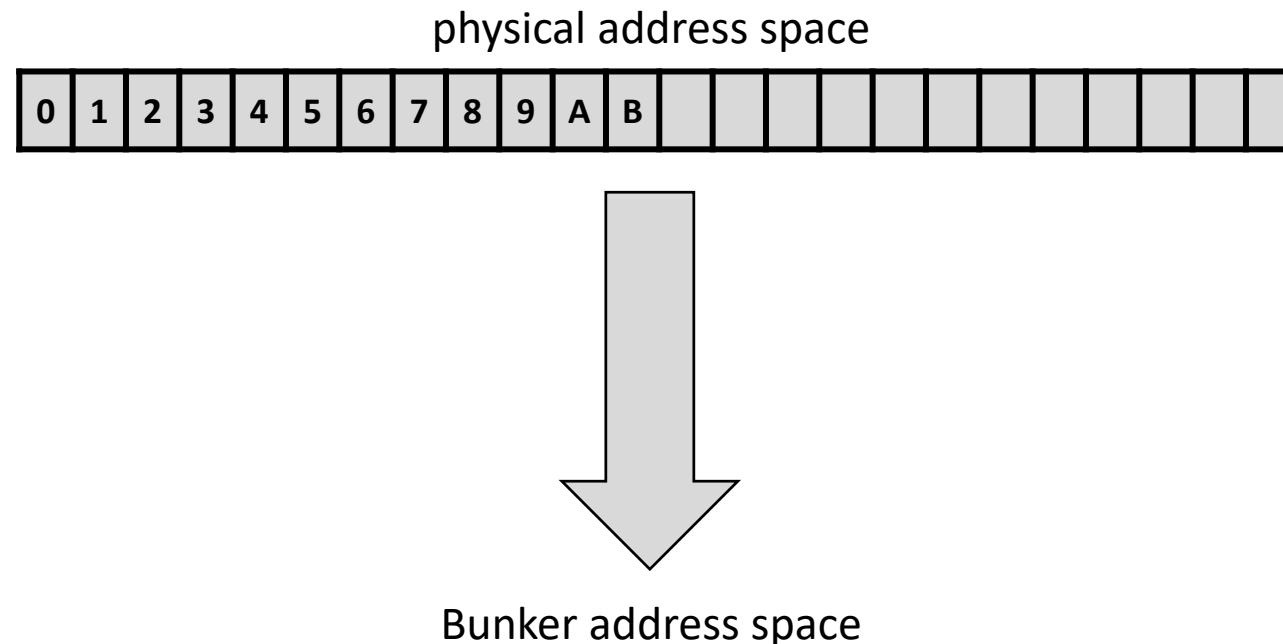
- Employs **similarity mapping**: virtual address translation within the cache
- + Saves miss latency and storage space for $X + \text{STRIDE}$



Approximate Spatio-Value Similarity

Similarity mapping:

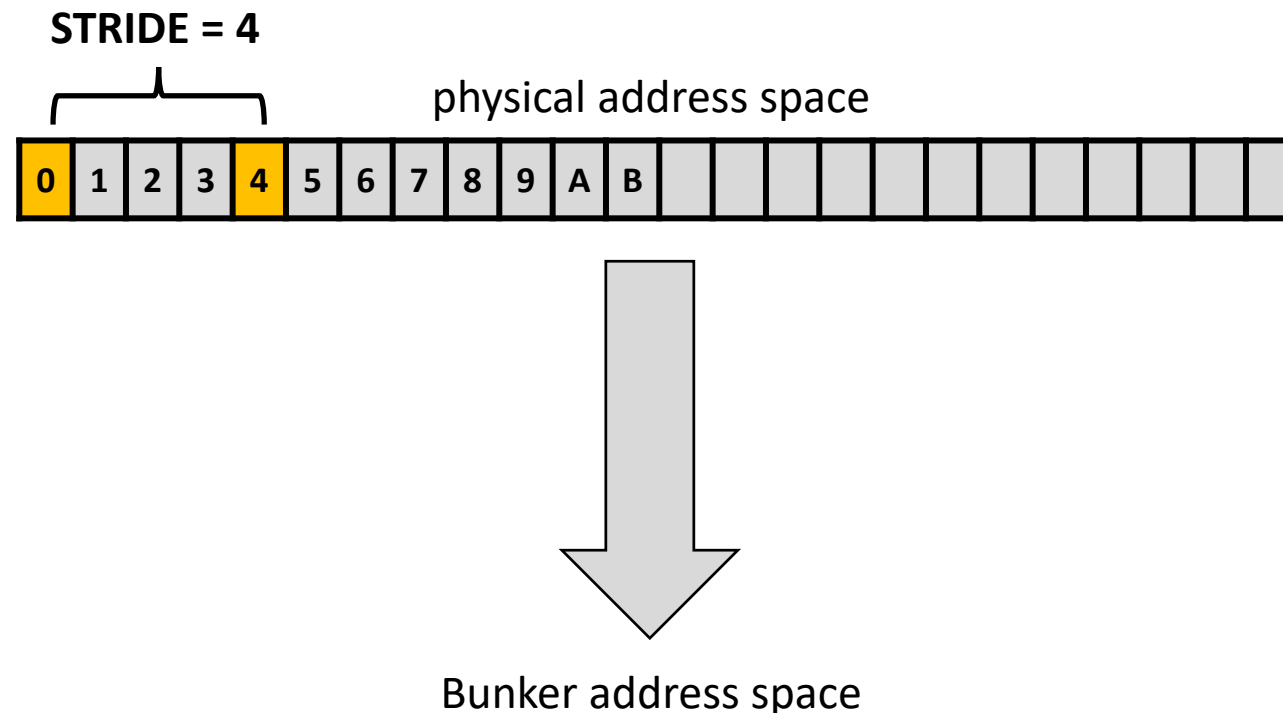
- Parameters are either user-provided or dynamically inferred
- **STRIDE**: distance between approximately similar blocks
- **RADIX**: degree (i.e., aggressiveness) of approximation



Approximate Spatio-Value Similarity

Similarity mapping:

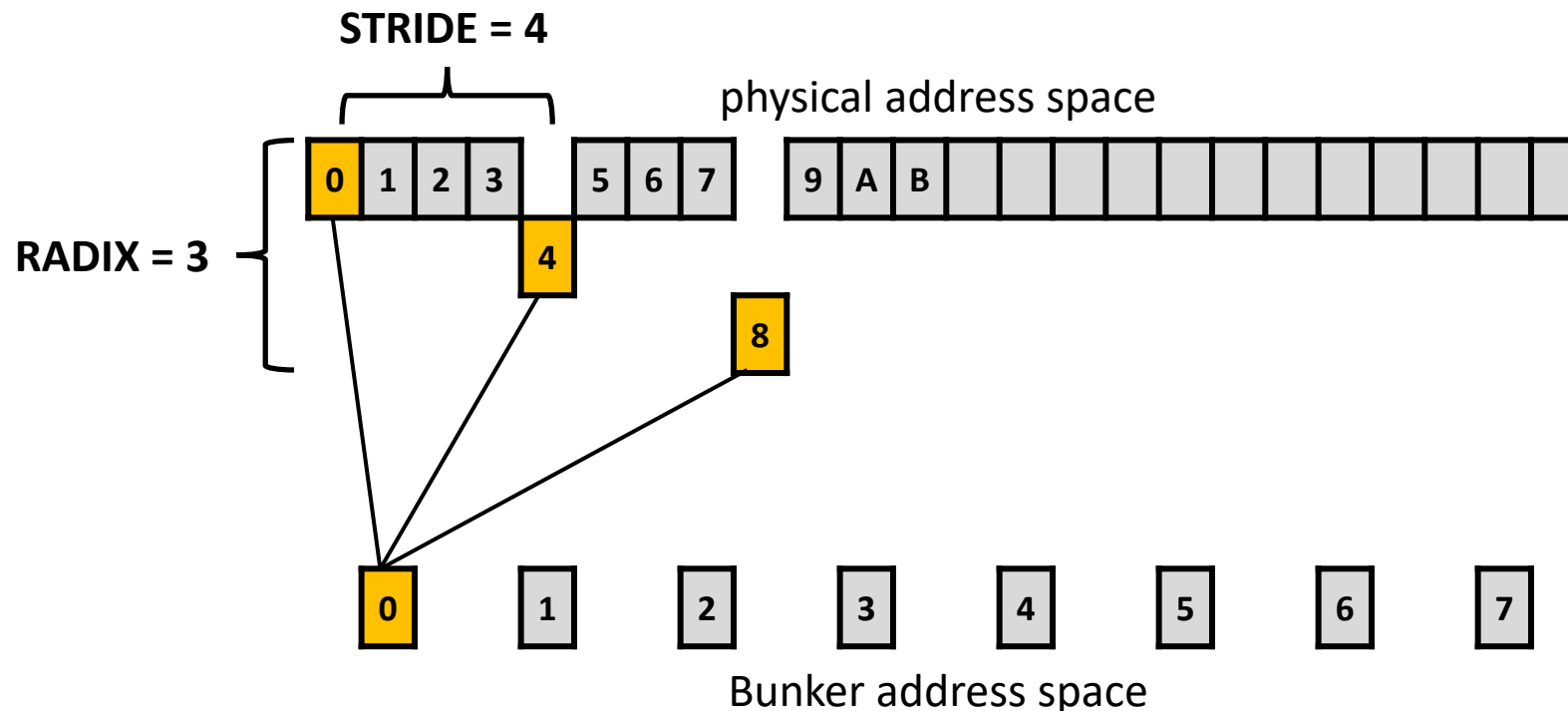
- Parameters are either user-provided or dynamically inferred
- STRIDE**: distance between approximately similar blocks
- RADIX**: degree (i.e., aggressiveness) of approximation



Approximate Spatio-Value Similarity

Similarity mapping:

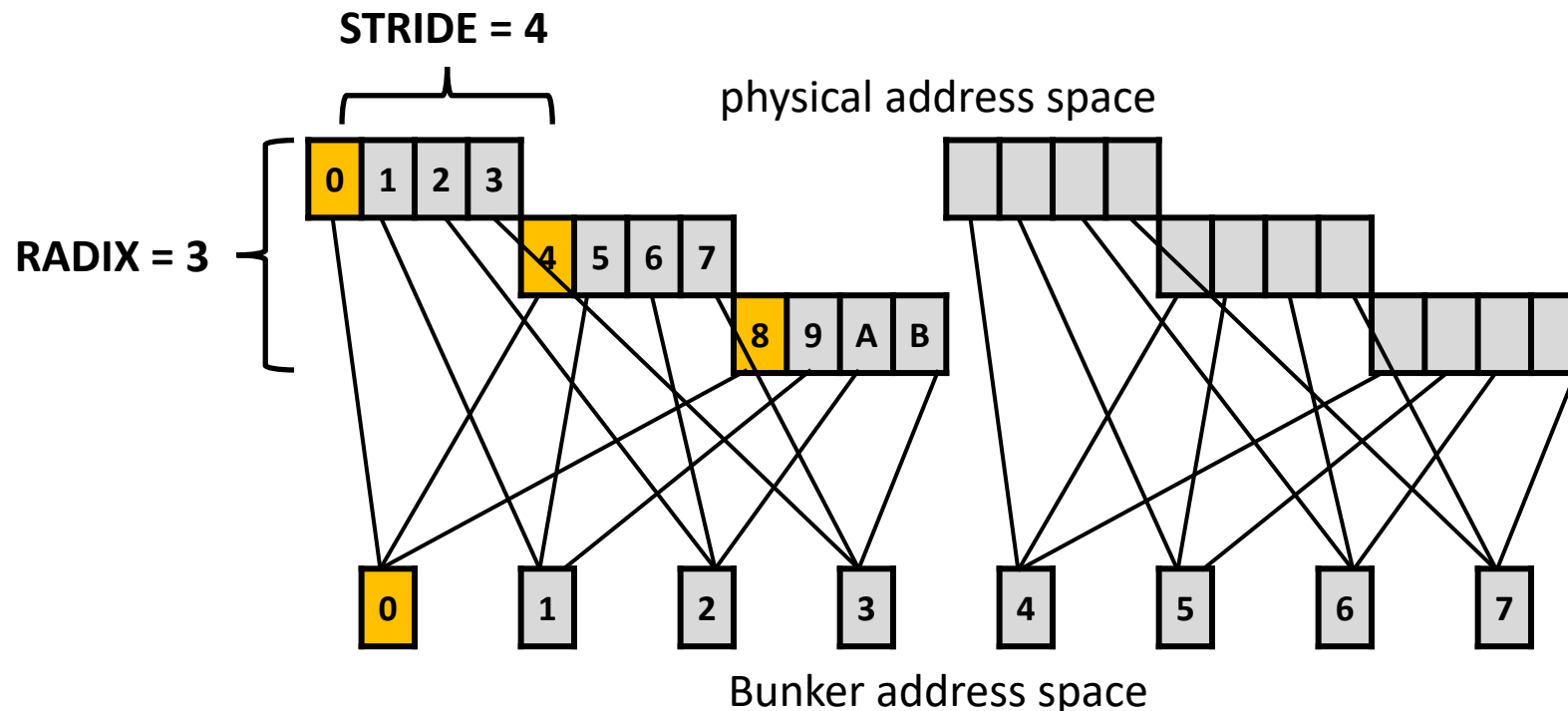
- Parameters are either user-provided or dynamically inferred
- STRIDE**: distance between approximately similar blocks
- RADIX**: degree (i.e., aggressiveness) of approximation



Approximate Spatio-Value Similarity

Similarity mapping:

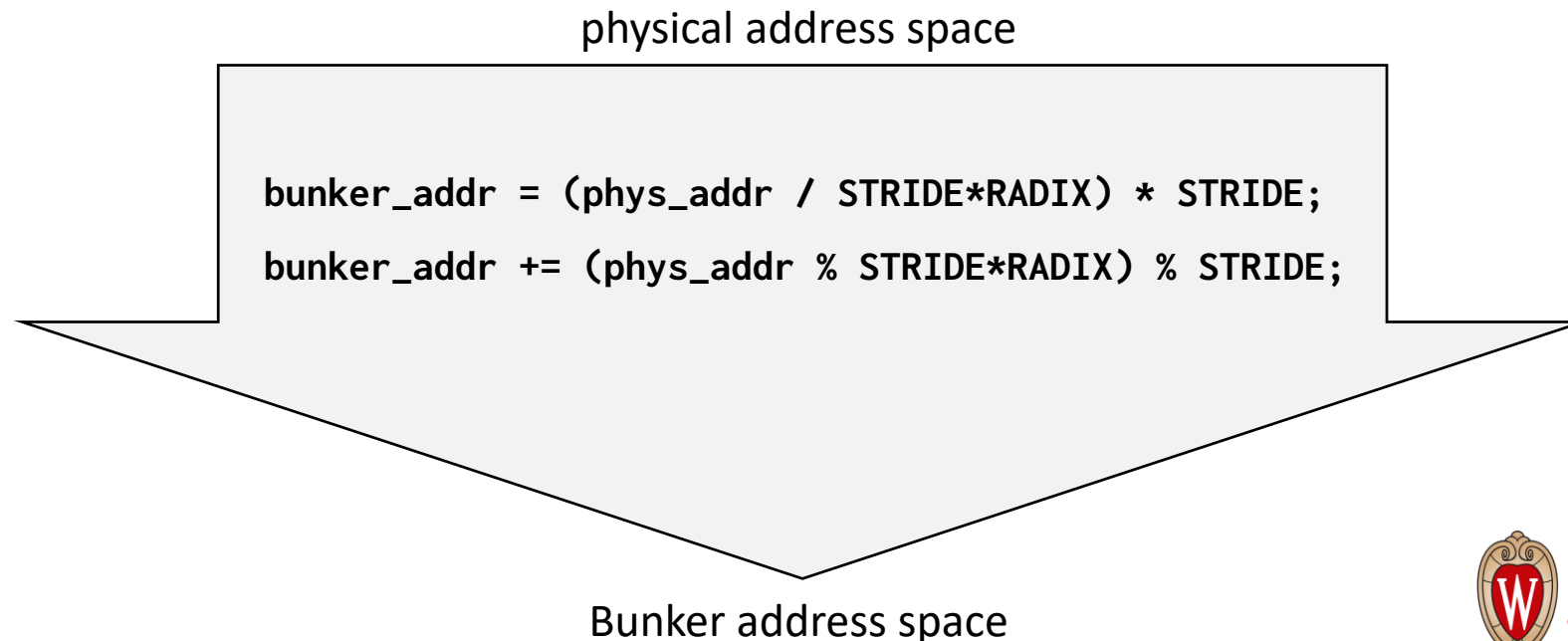
- Parameters are either user-provided or dynamically inferred
- STRIDE**: distance between approximately similar blocks
- RADIX**: degree (i.e., aggressiveness) of approximation



Approximate Spatio-Value Similarity

Similarity mapping:

- Parameters are either user-provided or dynamically inferred
- **STRIDE**: distance between approximately similar blocks
- **RADIX**: degree (i.e., aggressiveness) of approximation



Approximate Spatio-Value Similarity

Another way to think about it:

- + Cache prefetching **without fetching**
 - A request for address X implicitly prefetches $X + \text{STRIDE}$
- + Cache compression **without reading values**
 - Data array entry for $X + \text{STRIDE}$ is deduplicated without needing $[X + \text{STRIDE}]$